

DST Airlines

Membre de l'équipe

Nicolas CALO

Johan CLOOS

Younes Es-soualhi

Rathana LAT

Youssef ZNATI

Table des matières

Membre de l'équipe	1
Table des matières	2
Étape 1 - Récolte des données	6
Sources retenues	6
Wikipédia	7
Utilité	7
Méthode de collecte	7
URL	7
Exemple de données en entrée	7
Extrait de données tabulaires formatées après web scraping	8
Données utiles de la source	8
Github IP2Location	9
URL	9
Extrait du fichier stocké sur Github	9
API Air France KLM	10
URL	10
Caractéristiques de l'API	10
Exemple de données renvoyées par l'API	10
Contraintes et solutions implémentées liées à l'utilisation de l'API	13
Quantité de données récoltées	13
Au 8 décembre 2025	13
Sources non-retenues	13
API Lufthansa	14
URL	15
Contraintes	15
Anspperformance	15
Présentation :	15
Sources officielles data:	15
URL	15
Contraintes :	15
Extrait de données:	15
- https://ansperformance.eu/csv/	15
Données utiles de la source :	15
BTS (Bureau of Transportation Statics)	20
URL	20
Contraintes :	20
- Le téléchargement est long à se lancer et se télécharge par mois alors que la plage temporelle va de 1987 à 2025	20
Avantages :	20
URL	20
Zone géographique couverte :	20
Extrait de données : CSV	20

Données utiles de la source :	21
Opensky-network	26
Description	26
URL	26
Contraintes :	26
Example data:	26
OpenWeatherMap	27
URL	27
Description	27
Étape 2 - Architecture des données	28
Base de données relationnelle	28
Diagramme UML	29
Fichiers de requêtes SQL	30
Script de création de table de référence statique	30
Script de création de table des données de vol extraite de Mongoddb	31
Création de vues	33
Requête fonctionnelle sur les vues	35
Exemple d'une requête avec les vols passés au départ d'Ile-de-France	38
Conclusion de la base de données relationnelle	38
Données utiles de la source	40
Objectif de la sélection	40
Structure générale des données renvoyées par l'API Air France KLM	40
Données retenues	40
Récapitulatif	41
Catégorie : Identification du vol	41
Catégorie : Informations horaires	42
Catégorie : Localisation / Aéroports	42
Catégorie : Statut du vol et indicateurs	44
Arborescence des données retenue	45
Schema de l'objet stocké	46
Les Outils d'interactions avec la BDD	52
Étape 3 - Consommation de la donnée	55
Etude des données tabulaires	55
Contexte	55
Structuration des données	55
Variables explicatives	55
Variables descriptives	56
Variable cible	57
Variables non retenues	57
Analyse des catégories	57
Exploration des données	60
Contexte	60
Analyse de la variable flightStatusPublic	60
Analyse de la variable flightLegs_serviceType	61

Analyse de la variable flightLegs_status	62
Synthèse des irrégularités par flightStatusPublic	62
Synthèse des irrégularités par flightLegs_status	63
Synthèse des irrégularités par flightLegs_serviceType	63
Top 10 des flightLegs_aircraft_ownerAirlineCode par taux d'irrégularités	64
Top 10 des flightLegs_aircraft_typeCode par taux d'irrégularités	64
Pipeline de Machine Learning avec scikit-learn	66
Objectif	66
Jeu de données	66
Sélection des données	66
Feature engineering	66
Feature preprocessing	67
Statut du vol (classification)	67
Séparation jeux de données	67
Type de données	67
Hyperparameter tuning	67
Algorithmes	67
Exemple de Matrice de confusion	68
Conclusion	68
Durée du retard (classification)	69
Séparation jeux de données	69
Type de données	69
Hyperparameter tuning	69
Algorithmes	69
Exemple de Matrice de confusion	69
Conclusion	70
Durée du retard (régression)	71
Séparation jeux de données	71
Type de données	71
Hyperparameter tuning	71
Algorithmes	71
Conclusion	72
Étape 4 - Déploiement	73
Déploiement en local - Docker COMPOSE	73
Structure des services et dépendances du Docker Compose	74
API - Apprentissage Machine	75
API - PostgreSQL	76
API - mongoDB	77
Architecture du code de l'API :	78
Interface utilisateur Dash	79
Environnement GCP	81
Stockage des données – Google Cloud Storage	81
Traitements automatisés – Cloud Run	81
Fonctions Cloud Run	81

Cloud Run	82
Gestion des images – Cloud Build - Artifact Registry	83
Orchestration avancée – Google Kubernetes Engine (GKE)	83
Problèmes rencontrés lors du déploiement	83
Architecture du projet sur GCP	84
Étape 5 - Automatisation et Monitoring	85
Tableau de bord pour le monitoring des composants GKE :	85
Logs fonction d'extractions :	86
Logs fonctions de peuplement de la base de données mongoDB	86
logs transformation des données dans postgresQL	87
Logs API Mongodb	87
Logs API machine learning	88
Annexe	89
Diagramme UML complet généré avec Dbeaver	89

Étape 1 - Récolte des données

Sources retenues

Source	Type de données	Utilité	Méthode de récupération
Wikipedia https://www.wikiwand.com/en/articles/List_of_international_airports_by_country	Détails géographiques des aéroports mondiaux	Enrichissement du jeu de données de vols avec la localisation (continent, région, etc.)	Web scraping (Beautiful Soup)
Github IP2Location https://github.com/ip2location/ip2location-iata-icao	Correspondance code IATA/ICAO des aéroports	Nécessaire pour faire la correspondance entre les données Wikipédia et Air France KLM	Téléchargement direct du .csv mis à disposition
API Air France KLM https://developer.airfranceklm.com/	Informations historiques et prévisionnelles sur les vols Air France KLM	Données historiques pour entraîner l'apprentissage machine et description des vols futurs sur lesquels faire des prédictions	Script python interagissant avec l'API

Wikipédia

Utilité

- Enrichissement du jeu de données de vols avec la localisation (continent, région, etc.)



Méthode de collecte

- Script Python pour Web scraping (Beautiful Soup)

URL

- https://www.wikiwand.com/en/articles/List_of_international_airports_by_country

Exemple de données en entrée

^ Africa

Northern Africa

Algeria

More information Location, Airport ...			^
Location	Airport	IATA Code	
Algiers	Houari Boumediene Airport	ALG	^
Annaba	Rabah Bitat Airport	AAE	
Batna	Batna Airport	BLJ	
Béjaïa	Soummam Airport	BJA	
Chlef	Chlef International Airport	CFK	
Constantine	Mohamed Boudiaf International Airport	CZL	
Jijel	Jijel Ferhat Abbas Airport	GJL	
Oran	Oran Es Senia Airport	ORN	
Oumache	Biskra Airport	BSK	
Sétif	Ain Arnat Airport	QSF	
Tlemcen	Zenata – Messali El Hadj Airport	TLM	
Close			^

Extrait de données tabulaires formatées après web scraping

	1	2	3	4	5	6	7	8	9	10
	index	continent	subcontinent	country	Location	Aiport	IATA Code	ICAO Code	State/	Airport
2	0	Africa	Northern Africa	Algeria	Algiers		ALG			Houari Boumediene Airport
3	1	Africa	Northern Africa	Algeria	Annaba		AAE			Rabah Bitat Airport
4	2	Africa	Northern Africa	Algeria	Batna		BLJ			Batna Airport
5	3	Africa	Northern Africa	Algeria	Béjaïa		BJA			Soummam Airport
6	4	Africa	Northern Africa	Algeria	Chlef		CFK			Chlef International Airport
7	5	Africa	Northern Africa	Algeria	Constantine		CZL			Mohamed Boudiaf International Airport
8	6	Africa	Northern Africa	Algeria	Jijel		GJL			Jijel Ferhat Abbas Airport
9	7	Africa	Northern Africa	Algeria	Oran		ORN			Oran Es Senia Airport
10	8	Africa	Northern Africa	Algeria	Oumache		BSK			Biskra Airport
11	9	Africa	Northern Africa	Algeria	Sétif		QSF			Ain Arnat Airport
12	10	Africa	Northern Africa	Algeria	Tlemcen		TLM			Zenata – Messali El Hadj Airport
13	11	Africa	Northern Africa	Egypt	Alexandria		HBE			Borg El Arab International Airport
14	12	Africa	Northern Africa	Egypt	Alexandria		ALY			El Nouzha Airport (OPEN)
15	13	Africa	Northern Africa	Egypt	Asyut		ATZ			Assiut Airport
16	14	Africa	Northern Africa	Egypt	Aswan		ASW			Aswan International Airport
17	15	Africa	Northern Africa	Egypt	Cairo		CAI			Cairo International Airport
18	16	Africa	Northern Africa	Egypt	Cairo		SPX			Sphinx International Airport
19	17	Africa	Northern Africa	Egypt	El Arish		AAC			El Arish International Airport
20	18	Africa	Northern Africa	Egypt	El Dabaa		DBB			El Alamein International Airport
21	19	Africa	Northern Africa	Egypt	Hurghada		HRG			Hurghada International Airport
22	20	Africa	Northern Africa	Egypt	Luxor		LXR			Luxor International Airport
23	21	Africa	Northern Africa	Egypt	Marsa Alam		RMF			Marsa Alam International Airport
24	22	Africa	Northern Africa	Egypt	Mersa Matruh		MUH			Mersa Matruh International Airport
25	23	Africa	Northern Africa	Egypt	Saint Catherine		SKV			St. Catherine International Airport

Données utiles de la source

- Continent
- Sous-continent
- Pays
- Ville
- Nom de l'aéroport
- Code IATA

Github IP2Location

URL

- <https://github.com/ip2location/ip2location-iata-icao/blob/master/iata-icao.csv>



Extrait du fichier stocké sur Github

 ip2location Update iata-icao.csv 4f51d8d · 2 months ago History

PreviewCodeBlame

RawCopyDownloadEditDropdownOpen

```
1  "country_code","region_name","iata","icao","airport","latitude","longitude"
2  "AE","Abu Zaby","AAN","OMAL","Al Ain International Airport","24.2617","55.6092"
3  "AE","Abu Zaby","AUH","OMAA","Abu Dhabi International Airport","24.433","54.6511"
4  "AE","Abu Zaby","AYM","","Yas Island Seaplane Base","24.467","54.6103"
5  "AE","Abu Zaby","AZI","OMAD","Al Bateen Executive Airport","24.4283","54.4581"
6  "AE","Abu Zaby","DHF","OMAM","Al Dhafra Air Base","24.2482","54.5477"
7  "AE","Abu Zaby","XSB","OMBY","Sir Bani Yas Airport","24.2836","52.5803"
8  "AE","Abu Zaby","ZDY","OMDL","Dalma Airport","24.51","52.3352"
9  "AE","Al Fujayrah","FJR","OMFJ","Fujairah International Airport","25.1122","56.324"
10 "AE","Ash Shariqah","SHJ","OMSJ","Sharjah International Airport","25.3286","55.5172"
11 "AE","Dubayy","DCG","","Dubai Creek Seaplane Base","25.2422","55.3314"
12 "AE","Dubayy","DJH","","Jebel Ali Seaplane Base","24.989","55.0238"
13 "AE","Dubayy","DWC","OMDW","Al Maktoum International Airport","24.8964","55.1614"
14 "AE","Dubayy","DXB","OMDB","Dubai International Airport","25.2528","55.3644"
15 "AE","Dubayy","NHD","OMDM","Al Minhad Air Base","25.0268","55.3662"
16 "AE","Ra's al Khaymah","RHR","","Al Hamra Seaplane Base","25.691","55.778"
17 "AE","Ra's al Khaymah","RKT","OMRK","Ras Al Khaimah International Airport","25.6135","55.9388"
18 "AF","Badakhshan","DAZ","OADZ","Darwaz Airport","38.4611","70.8825"
19 "AF","Badakhshan","FBD","OAFZ","Fayzabad Airport","37.1211","70.5181"
20 "AF","Badakhshan","KUR","OARZ","Razer Airport","37.752","-89.0154"
21 "AF","Badakhshan","KWH","OAHN","Khwhan Airport","37.883","70.217"
22 "AF","Badakhshan","SGA","OASN","Sheghnan Airport","37.567","71.5"
23 "AF","Badghis","LQN","OAQN","Qala i Naw Airport","34.985","63.1178"
```

API Air France KLM



URL

- <https://developer.airfranceklm.com/>

Caractéristiques de l'API

- API acceptant les requêtes de type GET avec paramètres sur les critères de vols à renvoyer
- Authentification nécessaire avec clé API personnelle gratuite mais limitée
- Réponse renvoyée sous format .json contenant les données de vols sous forme de documents

Exemple de données renvoyées par l'API

Fichiers au format json (exemple d'un document parmi la collection) :

```
{
  "operationalFlights": [
    {
      "flightNumber": 150,
      "flightScheduleDate": "2025-10-09",
      "id": "20251009+EK+0150",
      "haul": "LONG",
      "route": [
        "AMS",
        "DXB"
      ],
      "airline": {
        "code": "EK",
        "name": "EMIRATES"
      },
      "flightRelations": {
        "previousFlightData": {
          "id": "20251009+EK+0149",
          "flightScheduleDate": "2025-10-09",
          "airlineCode": "EK",
          "flightNumber": "0149"
        }
      },
      "flightStatusPublic": "ON_TIME",
      "flightStatusPublicLangTransl": "On time",
      "flightLegs": [
        {
          "status": "S",
          "statusName": "Scheduled",
          "publishedStatus": "ONTIME",
          "departureInformation": {
            "airport": {
              "code": "AMS",

```

```
"name": "SCHIPHOL AIRPORT",
"nameLangTranl": "Amsterdam Schiphol Airport",
"city": {
  "code": "AMS",
  "name": "AMSTERDAM",
  "nameLangTranl": "Amsterdam",
  "country": {
    "areaCode": "N",
    "code": "NL",
    "name": "NETHERLANDS",
    "nameLangTranl": "Netherlands",
    "euroCountry": "Y",
    "euCountry": "Y"
  }
},
"location": {
  "latitude": 52.3083,
  "longitude": 4.7681
},
"places": {
  "gateNumber": [
    "G09"
  ],
  "parkingPosition": "G09",
  "pierCode": "G",
  "terminalCode": "03",
  "boardingPier": "G",
  "checkInZone": [
    "29"
  ],
  "boardingContactType": "C",
  "parkingPositionType": "C",
  "paxDepartureGate": []
},
"times": {
  "scheduled": "2025-10-09T21:50:00.000+02:00",
  "latestPublished": "2025-10-09T21:50:00.000+02:00"
},
"arrivalInformation": {
  "airport": {
    "code": "DXB",
    "name": "DUBAI INTERNATIONAL AIRPORT",
    "nameLangTranl": "Dubai International Airport",
    "city": {
      "code": "DXB",
      "name": "DUBAI",
      "nameLangTranl": "Dubai",
      "country": {
        "areaCode": "I",
        "code": "AE",
        "name": "UNITED ARAB EMIRATES",
        "nameLangTranl": "United Arab Emirates",
        "euroCountry": "N",
        "euCountry": "N"
      }
    }
  }
},
```

```

        "location": {
            "latitude": 25.25,
            "longitude": 55.3578
        },
        "places": {
            "gateNumber": [],
            "baggageBelt": [],
            "arrivalHall": []
        }
    },
    "times": {
        "scheduled": "2025-10-09T23:50:00.000+04:00",
        "latestPublished": "2025-10-09T23:50:00.000+04:00",
        "estimated": {
            "value": "2025-10-09T23:50:00.000+04:00"
        }
    }
},
"legStatusPublic": "ON_TIME",
"legStatusPublicLangTransl": "On time",
"passengerCustomsStatus": "NOT SCHENGEN",
"serviceType": "J",
"serviceTypeName": "Normal Service",
"restricted": false,
"completionPercentage": "0",
"timeZoneDifference": "+0200",
"aircraft": {
    "registration": "A6EEQ",
    "typeCode": "388",
    "typeName": "AIRBUS A380-800 PASSENGER",
    "physicalPaxConfiguration": "F014J076Y429",
    "physicalFreightConfiguration": "PP000LL000"
},
"irregularity": {
    "cancelled": "N",
    "delayCode": [],
    "delayReason": [],
    "delayDuration": [],
    "delayReasonCodePublic": [],
    "delayReasonPublicLangTransl": [],
    "delayReasonPublic": [],
    "delayInformation": []
}
}
]
}
],
"page": {
    "pageSize": 1,
    "pageNumber": 0,
    "fullCount": 1,
    "pageCount": 1,
    "totalPages": 1
}
}

```

Contraintes et solutions implémentées liées à l'utilisation de l'API

Contrainte technique	Solution
Détermination de l'étendue du jeu de données à récupérer	- 1^{re} collecte exploratoire (tous les vols sur 7 jours) afin d'estimer le volume de données total et les itinéraires principaux
100 appels par jour par clé API	- Roulement des clés API suivant une liste fournie par l'utilisateur - Logging du nombre d'appels journaliers déjà effectués - Limitation de la collecte aux départs/arrivées des 30 plus gros aéroports européens
Pagination des résultats limitée à 100 vols par réponse API	- Détermination automatique du nombre de pages par requête et itération des requêtes sur ce nombre
Paramétrisation des requêtes API (départ, destination, date, etc.)	- Liste de paramètres à renseigner dans un fichier .csv annexe
Consignation des requêtes déjà effectuées	- Comparaison entre la requête/page demandée et les fichiers déjà récupérés - Annotation automatique dans le fichier .csv de l'avancée de la récolte
Taille des données à stocker	- Compression des fichiers de sortie - Suppression automatique des fichiers obsolètes
Vols disponibles jusqu'à 6 mois dans le passé et 1 an dans le futur	- Actualisation automatique des dates à récupérer dans le fichier .csv des requêtes à effectuer
Actualisation des données de vols prévus et passé	- Téléchargement journalier des données des nouveaux vols passé/futurs et actualisation des vols du lendemain

Quantité de données récoltées

Au 8 décembre 2025

> 1'000'000 de vols historiques et 240'000 vols futurs récoltés

Sources non-retenues

Source	Type de données	Utilité potentielle	Raison d'abandon
API Luftansa https://developer.lufthansa.com	Informations historiques et prévisionnelles sur les vols Lufthansa	Données pour entraîner le machine learning	Inscription à l'API non fonctionnelle
Open Meteo https://open-meteo.com/	Données historiques et prédictions météo	Croisement des données de vols avec la météo au départ, à l'arrivée et sur le parcours	Après examen du jeu de données d'Air France KLM, la météo comptait pour moins de 0.5% des raisons de retard de vol, ce qui limite le ratio effort/bénéfice pour l'utilisation des données météo dans le projet.
OpenWeatherMap https://openweathermap.org/			
OpenSky API https://opensky-network.org/data/api	Données historiques sur les vols	Source de données alternative ou complémentaire à l'API d'Air France KLM	Principalement des données agrégées. Probablement un projet complet pour l'exploitation de cette base de données
Ansperformance https://ansperformance.eu/data/	Données historiques agrégées sur les vols et leurs retards en Europe	Meilleure connaissance de causes potentielles de retard et de quelles sources à inclure	Pas d'utilité directe pour la prédiction de retard sur un vol donné
BTS (Bureau of Transportation Statics) https://www.bts.gov/	Données historiques agrégées sur les vols et leurs retards aux USA		
EDI-GLA https://edi-gla.co.uk/	Base de données participative sur les itinéraires mondiaux de vols commerciaux	Source de données permettant potentiellement d'identifier les itinéraires les plus fréquentés	Plus utile une fois les données d'Air France KLM récupérées

API Lufthansa

URL

- <https://developer.lufthansa.com/member/register>

Contraintes

- Le formulaire d'inscription à l'API ne renvoie pas d'email de confirmation et il est donc impossible de se connecter à celle-ci

Ansperformance

Présentation :

Ansperformance propose des données intéressantes pour déterminer les proportions des causes de retard sur des totaux de vols. Comme par exemple le cumul d'un retard en minute à cause de la météo pour un jour donné, le cumul de retard à cause, le cumul de retard à cause de grève, à cause de problème d'équipement, panne etc...

Sources officielles data:

- [EUROCONTROL Aviation Sustainability Unit](#).

URL

- <https://ansperformance.eu/data/>

Contraintes :

- Fichiers téléchargeables en csv mais pas d'api.

Avantages:

- On peut facilement importer les csv dans de la bdd relationnelle.
- Données historiques des retards par aéroport et jour

Extrait de données:

- <https://ansperformance.eu/csv/>
- **Tranche temporelle:** 2011-2025
- **Zone géographique:** Europe
- Airport Arrival ATFM Delays (from 2014)
- Daily en-route delays

Données utiles de la source :

Nom du champ	Type	Unité	Catégorie	Description en français	Exemple
YEAR	Entier	Année	Temporel	Année de référence	2015
MONTH_NUM	Entier	Mois (1-12)	Temporel	Numéro du mois	3
MONTH_MON	Texte	Code 3 lettres	Temporel	Mois (code à 3 lettres)	MAR
FLT_DATE	Date	JJ/MM/AAAA	Temporel	Date du vol	17/03/2015
ENTITY_NAME	Texte	-	Identification	Nom de l'entité (ANSP, FAB ou zone géographique)	FAB CE
ENTITY_TYPE	Texte	-	Identification	Type d'entité : ANSP (fournisseur), FAB (bloc d'espace) ou AREA (zone)	FAB (AUA)
FLT_ERT_1	Entier	Nombre de vols	Trafic	Nombre total de vols dans l'espace aérien ce jour	3853
DLY_ERT_1	Entier	MINUTES	Retard total	Minutes totales de retard ATFM en route (somme de tous les retards)	4262
DLY_ERT_A_1	Entier	MINUTES	Retard - Perturbation	Minutes de retard ATFM - Code A :	0

				Accident ou incident	
DLY_ERT_C_1	Entier	MINUTES	Retard - Capacité ATC	Minutes de retard ATFM - Code C : Capacité ATC insuffisante	1724
DLY_ERT_D_1	Entier	MINUTES	Retard - Météo	Minutes de retard ATFM - Code D : Opérations de dégivrage	0
DLY_ERT_E_1	Entier	MINUTES	Retard - Perturbation	Minutes de retard ATFM - Code E : Panne d'équipement (non-ATC)	0
DLY_ERT_G_1	Entier	MINUTES	Retard - Capacité	Minutes de retard ATFM - Code G : Capacité aéroportuaire insuffisante	0
DLY_ERT_I_1	Entier	MINUTES	Retard - Perturbation ATC	Minutes de retard ATFM - Code I : Grève du personnel ATC	0
DLY_ERT_M_1	Entier	MINUTES	Retard - Capacité	Minutes de retard ATFM - Code M : Problèmes de gestion de l'espace aérien	0

DLY_ERT_N_1	Entier	MINUTES	Retard - Perturbation	Minutes de retard ATFM - Code N : Grève du personnel non-ATC	0
DLY_ERT_O_1	Entier	MINUTES	Retard - Perturbation	Minutes de retard ATFM - Code O : Autres causes non spécifiées	378
DLY_ERT_P_1	Entier	MINUTES	Retard - Événement	Minutes de retard ATFM - Code P : Événement spécial (sommets, défilé, etc.)	2160
DLY_ERT_R_1	Entier	MINUTES	Retard - Capacité	Minutes de retard ATFM - Code R : Reroutage imposé par l'ATC	0
DLY_ERT_S_1	Entier	MINUTES	Retard - Personnel ATC	Minutes de retard ATFM - Code S : Manque de personnel ATC (effectifs)	0
DLY_ERT_T_1	Entier	MINUTES	Retard - Perturbation ATC	Minutes de retard ATFM - Code T : Panne d'équipement ATC	184
DLY_ERT_V_1	Entier	MINUTES	Retard - Capacité	Minutes de retard ATFM - Code V : Restrictions environnementales	86

DLY_ERT_W_1	Entier	MINUTES	Retard - Météo	Minutes de retard ATFM - Code W : Conditions météorologiques défavorables	556
DLY_ERT_NA_1	Entier	MINUTES	Retard - Perturbation	Minutes de retard ATFM - Code NA : Cause non précisée ou non attribuée	0
FLT_ERT_1_DLY	Entier	Nombre de vols	Trafic retardé	Nombre de vols ayant subi au moins 1 minute de retard ATFM	414
FLT_ERT_1_DLY_15	Entier	Nombre de vols	Trafic retardé	Nombre de vols ayant subi plus de 15 minutes de retard ATFM	99

BTS (Bureau of Transportation Statics)

URL

- <https://www.bts.gov/>

Contraintes :

- Le téléchargement est long à se lancer et se télécharge par mois alors que la plage temporelle va de 1987 à 2025

Avantages :

- Parfait pour faire une grosse bdd historique aux usa. Très gros volume disponible.

The screenshot shows the Bureau of Transportation Statistics website. The main header is blue with the text 'Bureau of Transportation Statistics' and a search bar. Below the header are navigation links: 'Topics and Geography', 'Statistical Products and Data', 'National Transportation Library', 'Newsroom', and 'About BTS'. The main content area is titled 'On-Time : Reporting Carrier On-Time Performance (1987-present)' and shows the latest available data as June 2025. There are filters for Geography (All), Year (2025), and Period (January). A table of fields is displayed with columns for Field Name, Description, and Support Table. The fields include Time Period (Year, Quarter, Month, Day of Month, Day of Week, Flight Date) and Airline (Reporting_Airline). Each field has a 'Get Lookup Table' link. A 'Download' button is also present.

Field Name	Description	Support Table
Time Period		
☐ Year	Year	
☐ Quarter	Quarter (1-4)	Get Lookup Table
☐ Month	Month	Get Lookup Table
☐ Day of Month	Day of Month	
☐ Day of Week	Day of Week	Get Lookup Table
☐ Flight Date	Flight Date (yyyymmdd)	
Airline		
☐ Reporting_Airline	Unique Carrier Code. When the same code has been used by multiple carriers, a numeric suffix is used for earlier users, for example, PA, PA(1), PA(2). Use this field for analysis across a range of years.	Get Lookup Table
An identification number assigned by U.S. DOT to identify a unique		

URL

https://www.transtats.bts.gov/DL_SelectFields.aspx?gnoyr_VQ=FGJ&QO_fu146_anzr=b0-gvzr

Zone géographique couverte :

- USA

Extrait de données : CSV

	P	Q	R	S	T	U	V	W	X	Y	Z	AA	AB	AC	AD	AE
1	DEP_DELAY	DEP_DELAY_NEW	CANCELLED	CANCELLATION_CODE	DIVERTED	CRS_ELAPSED_TIME	ACTUAL_ELAPSED_TIME	AIR_TIME	FLIGHTS	DISTANCE	CARRIER_DELAY	WEATHER_DELAY	NAS_DELAY	SECURITY_DELAY	LATE_AIRCRAFT_DELAY	
2	-15.00	0.00	0.00		0.00	126.00	110.00	93.00	1.00	685.00						
3	-5.00	0.00	0.00		0.00	107.00	95.00	78.00	1.00	481.00						
4	-4.00	0.00	0.00		0.00	107.00	98.00	78.00	1.00	481.00						
5	-10.00	0.00	0.00		0.00	241.00	210.00	197.00	1.00	1539.00						
6	61.00	61.00	0.00		0.00	167.00	168.00	146.00	1.00	1041.00	0.00	0.00	1.00	0.00	61.00	
7	-8.00	0.00	0.00		0.00	148.00	143.00	126.00	1.00	906.00						
8	-6.00	0.00	0.00		0.00	100.00	96.00	80.00	1.00	518.00						
9	-3.00	0.00	0.00		0.00	164.00	163.00	144.00	1.00	1018.00						
10	9.00	9.00	0.00		0.00	155.00	145.00	130.00	1.00	970.00						
11	-7.00	0.00	0.00		0.00	143.00	136.00	123.00	1.00	882.00						

Données utiles de la source :

Cause of Delay (Data starts 6/2003)

CarrierDelay	Carrier Delay, in Minutes	
WeatherDelay	Weather Delay, in Minutes	
NASDelay	National Air System Delay, in Minutes	
SecurityDelay	Security Delay, in Minutes	
LateAircraftDelay	Late Aircraft Delay, in Minutes	

Cancellations and Diversions

Cancelled	Cancelled Flight Indicator (1=Yes)	Get Lookup Table
CancellationCode	Specifies The Reason For Cancellation	Get Lookup Table
Diverted	Diverted Flight Indicator (1=Yes)	Get Lookup Table

Time Period

Year	Year	
Quarter	Quarter (1-4)	Get Lookup Table
Month	Month	Get Lookup Table
DayofMonth	Day of Month	

DayOfWeek	Day of Week	Get Lookup Table
FlightDate	Flight Date (yyyymmdd)	

Airline

Reporting_Airline	Unique Carrier Code. When the same code has been used by multiple carriers, a numeric suffix is used for earlier users, for example, PA, PA(1), PA(2). Use this field for analysis across a range of years.	Get Lookup Table
DOT_ID_Reporting_Airline	An identification number assigned by US DOT to identify a unique airline (carrier). A unique airline (carrier) is defined as one holding and reporting under the same DOT certificate regardless of its Code, Name, or holding company/corporation.	Get Lookup Table
IATA_CODE_Reporting_Airline	Code assigned by IATA and commonly used to identify a carrier. As the same code may have been assigned to different carriers over time, the code is not always unique. For analysis, use the Unique Carrier Code.	Get Lookup Table
Tail_Number	Tail Number	
Flight_Number_Reporting_Airline	Flight Number	

Destination

DestAirportID	Destination Airport, Airport ID. An identification number assigned by US DOT to identify a unique airport. Use this field for airport analysis across a range of years because an airport can change its airport code and airport codes can be reused.	Get Lookup Table
DestAirportsEqID	Destination Airport, Airport Sequence ID. An identification number assigned by US DOT to identify a unique airport at a given point of time. Airport attributes, such as airport name or coordinates, may change over time.	Get Lookup Table

DestCityMarketID	Destination Airport, City Market ID. City Market ID is an identification number assigned by US DOT to identify a city market. Use this field to consolidate airports serving the same city market.	Get Lookup Table
Dest	Destination Airport	Get Lookup Table
DestCityName	Destination Airport, City Name	
DestState	Destination Airport, State Code	Get Lookup Table
DestStateFips	Destination Airport, State Fips	Get Lookup Table
DestStateName	Destination Airport, State Name	
DestWac	Destination Airport, World Area Code	Get Lookup Table

Departure Performance

CRSDepTime	CRS Departure Time (local time: hhmm)	
DepTime	Actual Departure Time (local time: hhmm)	
DepDelay	Difference in minutes between scheduled and actual departure time. Early departures show negative numbers.	
DepDelayMinutes	Difference in minutes between scheduled and actual departure time. Early departures set to 0.	
DepDel15	Departure Delay Indicator, 15 Minutes or More (1=Yes)	Get Lookup Table
DepartureDelayGroups	Departure Delay intervals, every (15 minutes from <-15 to >180)	Get Lookup Table
DepTimeBlk	CRS Departure Time Block, Hourly Intervals	Get Lookup Table

TaxiOut	Taxi Out Time, in Minutes	
WheelsOff	Wheels Off Time (local time: hhmm)	

Arrival Performance

WheelsOn	Wheels On Time (local time: hhmm)	
TaxiIn	Taxi In Time, in Minutes	
CRSArrTime	CRS Arrival Time (local time: hhmm)	
ArrTime	Actual Arrival Time (local time: hhmm)	
ArrDelay	Difference in minutes between scheduled and actual arrival time. Early arrivals show negative numbers.	
ArrDelayMinutes	Difference in minutes between scheduled and actual arrival time. Early arrivals set to 0.	
ArrDel15	Arrival Delay Indicator, 15 Minutes or More (1=Yes)	Get Lookup Table
ArrivalDelayGroups	Arrival Delay intervals, every (15-minutes from <-15 to >180)	Get Lookup Table
ArrTimeBlk	CRS Arrival Time Block, Hourly Intervals	Get Looku

Flight Summaries

CRSElapsedTime	CRS Elapsed Time of Flight, in Minutes	
ActualElapsedTime	Elapsed Time of Flight, in Minutes	
AirTime	Flight Time, in Minutes	
Flights	Number of Flights	
Distance	Distance between airports (miles)	

DistanceGroup	Distance Intervals, every 250 Miles, for Flight Segment	Get Lookup Table
---------------	---	----------------------------------

Opensky-network

Description

Route et localisation en temps réel des avions

URL

- <https://opensky-network.org/api>

Contraintes :

- On a les vols dans le monde entier mais pas de données utiles pour la prévision des retards.
- Zone géographique couverte : monde
- Requêtes limitées par jours :

Surface (km²)	Crédits	Description
0 - 25 (<500×500 km)	1	Petite zone (ville)
25 - 100 (<1000×1000 km)	2	Région moyenne
100 - 400 (<2000×2000 km)	3	Grande région
>400 ou monde entier	4	Monde entier sans filtrage

Example data:

```
{
  "icao24": "008d3c",
  "callsign": "FSK900",
  "startTime": 1759951644,
  "endTime": 1759963372,
  "path": [
    [1759951644, -33.9658, 18.602, -304, 165, false],
    [1759951675, -33.9864, 18.6084, 0, 165, false],
    ...
  ]
}
```

OpenWeatherMap

URL

- <https://openweathermap.org/>

Description

Le site openweathermap ne permet pas de collecter les données historisées gratuitement. Pour pouvoir se faire, il faut s'inscrire avec un compte d'étudiant en renseignant son numéro d'étudiant et l'expiration de sa carte.

Nous n'avons pas encore de données météorologiques à présenter.

Nous prenons en compte cette source de données dans le cadre du projet dans la mesure du possible en essayant des API sur d'autres sites météorologiques.

Étape 2 - Architecture des données

Base de données relationnelle



L'objectif d'implémenter une base de données relationnelle est de permettre d'avoir une base structurée afin de faciliter le requêtage ou l'interaction avec les autres modules du projet comme le machine learning et l'interface utilisateur Dash.

Avec Postgresql, les tables des données de référence statiques ont été créées. L'alimentation de ces tables provient d'extractions de données par webscraping.

Nom des tables :

- Continent
- Subcontinent
- Country
- Location
- Airport

Les données stockées dans MongoDB sont des fichiers au format json provenant de l'API d'Air France KLM contenant les informations concernant les vols. Les tables des données exportées de MongoDB ont été créées.

Nom des tables :

- Flight
- Arrival
- Departure
- Delay

Diagramme UML

Un diagramme UML complet généré avec dbeaver est en annexe.

Les tables en bleu concernent les données de référence statique. Les tables en vert concernent les données extraites de MongoDB.

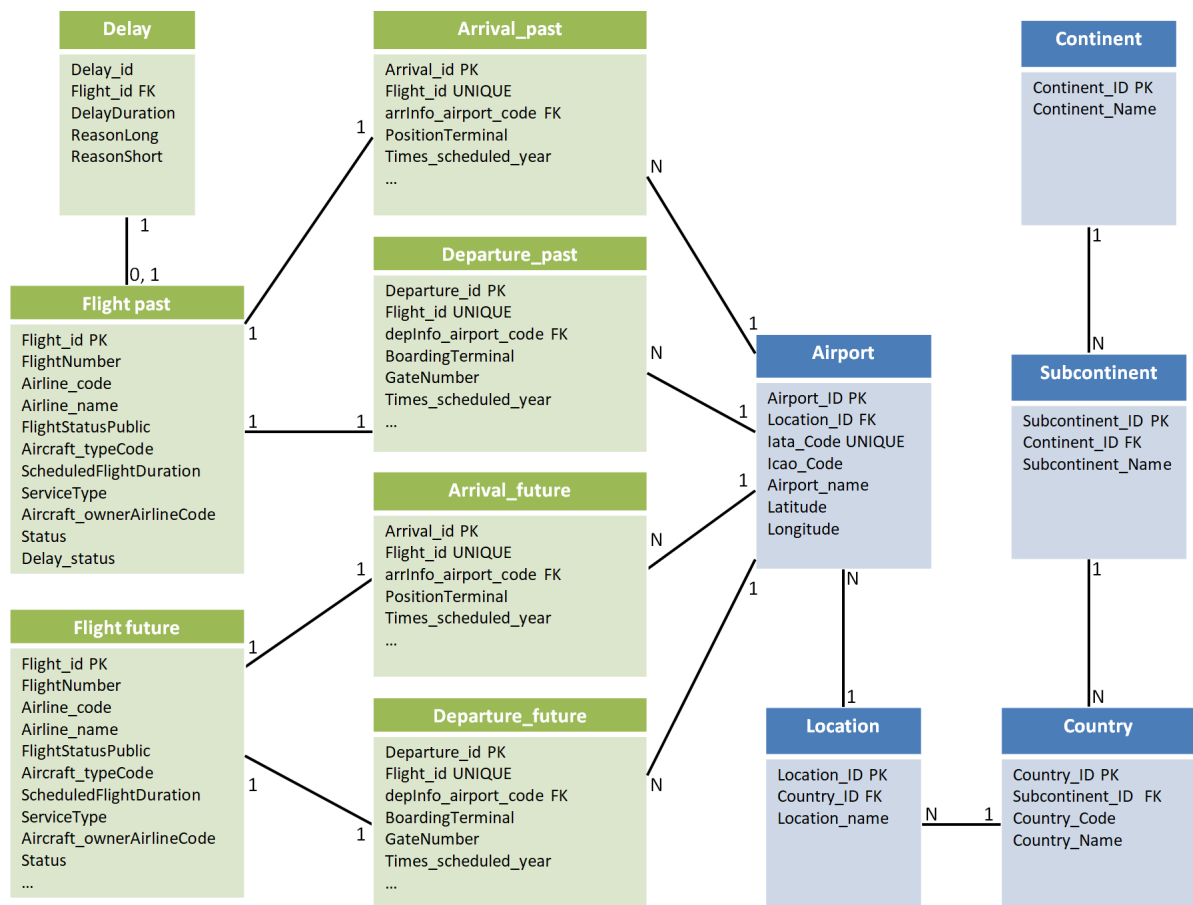
Les tables bleues contenant les informations géographiques ont une relation un à plusieurs (1, N).

- Un Continent a plusieurs Subcontinent.
- Un Subcontinent a plusieurs Country.
- Un Country a plusieurs Location.
- Un Location à plusieurs Airport. (Location_name inclut des villes ou des régions)
- Un Airport a plusieurs Departure_past, Arrival_past, Departure_future, Arrival_future.

Les tables vertes contiennent les informations du vol avec les relations :

- 1 ou 0 Flight_past peut avoir un Delay.
- Les relations sont de 1 à 1 pour :
 - Flight_past, Departure_past, Arrival_past
 - Flight_future, Departure_future, Arrival_future

La raison de ne pas les avoir mis dans la même table est de distinguer les informations du vol entre le passé, le futur, le départ et l'arrivée permettant d'avoir une meilleure compréhension sur la donnée.



Fichiers de requêtes SQL

Script de création de table de référence statique

```
DROP TABLE IF EXISTS Continent CASCADE;
CREATE TABLE Continent (
Continent_ID serial PRIMARY KEY,
Continent_Name varchar(50) NOT NULL
);
```

```
DROP TABLE IF EXISTS Subcontinent CASCADE;
CREATE TABLE Subcontinent (
Subcontinent_ID serial PRIMARY KEY,
Subcontinent_Name varchar(100) NOT NULL,
Continent_ID int NOT NULL,
FOREIGN KEY (Continent_ID) REFERENCES Continent(Continent_ID)
);
```

```
DROP TABLE IF EXISTS Country CASCADE;
CREATE TABLE Country (
Country_ID serial PRIMARY KEY,
Country_Name varchar(100) DEFAULT NULL,
Country_Code varchar(100) NOT NULL,
Subcontinent_ID int NOT NULL,
FOREIGN KEY (Subcontinent_ID) REFERENCES Subcontinent(Subcontinent_ID)
);
```

```
DROP TABLE IF EXISTS Location CASCADE;
CREATE TABLE Location (
Location_ID serial PRIMARY KEY,
Location_name varchar(100) NOT NULL,
Country_ID int NOT NULL,
FOREIGN KEY (Country_ID) REFERENCES Country(Country_ID)
);
```

```
DROP TABLE IF EXISTS Airport CASCADE;
CREATE TABLE Airport (
Airport_ID serial PRIMARY KEY,
Airport_Name varchar(100) NOT NULL,
Iata_Code varchar(3) UNIQUE NOT NULL,
Icao_Code varchar(4) NOT NULL,
Latitude float NOT NULL,
Longitude float NOT NULL,
Location_ID int NOT NULL,
FOREIGN KEY (Location_ID) REFERENCES Location(Location_ID)
);
```

Script de création de table des données de vol extraite de Mongoddb

```
DROP TABLE IF EXISTS flight_past CASCADE;
CREATE TABLE flight_past (
flight_id varchar(50) PRIMARY KEY,
flightNumber int DEFAULT NULL,
airline_code varchar(50) NULL,
airline_name varchar(100) NULL,
flightStatusPublic varchar(50) NULL,
flightLegs_aircraft_typeCode varchar(50) NULL,
flightLegs_scheduledFlightDuration varchar(50) NULL,
flightLegs_serviceType varchar(50) NULL,
flightLegs_aircraft_ownerAirlineCode varchar(50) NULL,
flightLegs_status varchar(100) NULL,
flightLegs_serviceTypeName varchar(100) NULL,
flightLegs_publishedStatus varchar(100) NULL,
flightLegs_legStatusPublic varchar(100) NULL,
flightLegs_statusName varchar(100) NULL,
delay_status boolean NULL
);
```

```
DROP TABLE IF EXISTS flight_future CASCADE;
CREATE TABLE flight_future (
flight_id varchar(50) PRIMARY KEY,
flightNumber int DEFAULT NULL,
airline_code varchar(50) NULL,
airline_name varchar(100) NULL,
flightStatusPublic varchar(50) NULL,
flightLegs_aircraft_typeCode varchar(50) NULL,
flightLegs_scheduledFlightDuration varchar(50) NULL,
flightLegs_serviceType varchar(50) NULL,
flightLegs_aircraft_ownerAirlineCode varchar(50) NULL,
flightLegs_status varchar(100) NULL,
flightLegs_serviceTypeName varchar(100) NULL,
flightLegs_publishedStatus varchar(100) NULL,
flightLegs_legStatusPublic varchar(100) NULL,
flightLegs_statusName varchar(100) NULL,
delay_status boolean NULL
);
```

```
DROP TABLE IF EXISTS delay CASCADE;
CREATE TABLE delay (
delay_id serial PRIMARY KEY,
flight_id varchar(50) UNIQUE NOT NULL,
flightLegs_irregularity_delayDuration varchar(250) NULL,
flightLegs_irregularity_delayDuration_total varchar(250) NULL,
flightLegs_irregularity_delayInfo_delayReasonPublicLong varchar(250) NULL,
flightLegs_irregularity_delayInfo_delayReasonPublicShort varchar(250) NULL,
FOREIGN KEY (flight_id) REFERENCES flight_past(flight_id)
);
```

```
DROP TABLE IF EXISTS departure_future CASCADE;
```

```

CREATE TABLE departure_future (
departure_id serial PRIMARY KEY,
flight_id varchar(50) UNIQUE NOT NULL,
flightLegs_deplInfo_airport_code varchar(3) NOT NULL,
flightLegs_deplInfo_airport_places_depPosTerm_boardingTerminal varchar(50) NULL,
flightLegs_deplInfo_airport_places_depPosTerm_gateNumber varchar(50) NULL,
flightLegs_deplInfo_times_scheduled_date date NULL,
flightLegs_deplInfo_times_scheduled_time time NULL,
flightLegs_deplInfo_times_scheduled_year int NULL,
flightLegs_deplInfo_times_scheduled_month int NULL,
flightLegs_deplInfo_times_scheduled_day int NULL,
flightLegs_deplInfo_times_scheduled_hour int NULL,
flightLegs_deplInfo_times_scheduled_minute int NULL,
flightLegs_deplInfo_times_scheduled_timezone varchar(6) NULL,
flightLegs_deplInfo_times_number_week int NULL,
FOREIGN KEY (flight_id) REFERENCES flight_future(flight_id),
FOREIGN KEY (flightLegs_deplInfo_airport_code) REFERENCES Airport(lata_Code)
);

```

```

DROP TABLE IF EXISTS departure_past CASCADE;
CREATE TABLE departure_past (
departure_id serial PRIMARY KEY,
flight_id varchar(50) UNIQUE NOT NULL,
flightLegs_deplInfo_airport_code varchar(3) NOT NULL,
flightLegs_deplInfo_airport_places_depPosTerm_boardingTerminal varchar(50) NULL,
flightLegs_deplInfo_airport_places_depPosTerm_gateNumber varchar(50) NULL,
flightLegs_deplInfo_times_scheduled_date date NULL,
flightLegs_deplInfo_times_scheduled_time time NULL,
flightLegs_deplInfo_times_scheduled_year int NULL,
flightLegs_deplInfo_times_scheduled_month int NULL,
flightLegs_deplInfo_times_scheduled_day int NULL,
flightLegs_deplInfo_times_scheduled_hour int NULL,
flightLegs_deplInfo_times_scheduled_minute int NULL,
flightLegs_deplInfo_times_scheduled_timezone varchar(6) NULL,
flightLegs_deplInfo_times_number_week int NULL,
FOREIGN KEY (flight_id) REFERENCES flight_past(flight_id),
FOREIGN KEY (flightLegs_deplInfo_airport_code) REFERENCES Airport(lata_Code)
);

```

```

DROP TABLE IF EXISTS arrival_future CASCADE;
CREATE TABLE arrival_future (
arrival_id serial PRIMARY KEY,
flight_id varchar(50) UNIQUE NOT NULL,
flightLegs_arrlInfo_airport_code varchar(3) NOT NULL,
flightLegs_arrlInfo_airport_places_arrivalPositionTerminal varchar(50) NULL,
flightLegs_arrlInfo_times_scheduled_date date NULL,
flightLegs_arrlInfo_times_scheduled_time time NULL,
flightLegs_arrlInfo_times_scheduled_year int NULL,
flightLegs_arrlInfo_times_scheduled_month int NULL,
flightLegs_arrlInfo_times_scheduled_day int NULL,
flightLegs_arrlInfo_times_scheduled_hour int NULL,
flightLegs_arrlInfo_times_scheduled_minute int NULL,

```



```

flightLegs_arrInfo_times_scheduled_timezone varchar(6) NULL,
flightLegs_arrInfo_times_number_week int NULL,
FOREIGN KEY (flight_id) REFERENCES flight_future(flight_id),
FOREIGN KEY (flightLegs_arrInfo_airport_code) REFERENCES Airport(lata_Code)
);

```

```

DROP TABLE IF EXISTS arrival_future_d1 CASCADE;
CREATE TABLE arrival_future_d1 (
arrival_id serial PRIMARY KEY,
flight_id varchar(50) UNIQUE NOT NULL,
flightLegs_arrInfo_airport_code varchar(3) NOT NULL,
flightLegs_arrInfo_airport_places_arrivalPositionTerminal varchar(50) NULL,
flightLegs_arrInfo_times_scheduled_date date NULL,
flightLegs_arrInfo_times_scheduled_time time NULL,
flightLegs_arrInfo_times_scheduled_year int NULL,
flightLegs_arrInfo_times_scheduled_month int NULL,
flightLegs_arrInfo_times_scheduled_day int NULL,
flightLegs_arrInfo_times_scheduled_hour int NULL,
flightLegs_arrInfo_times_scheduled_minute int NULL,
flightLegs_arrInfo_times_scheduled_timezone varchar(6) NULL,
flightLegs_arrInfo_times_number_week int NULL,
FOREIGN KEY (flight_id) REFERENCES flight_future_d1(flight_id),
FOREIGN KEY (flightLegs_arrInfo_airport_code) REFERENCES Airport(lata_Code)
);

```

```

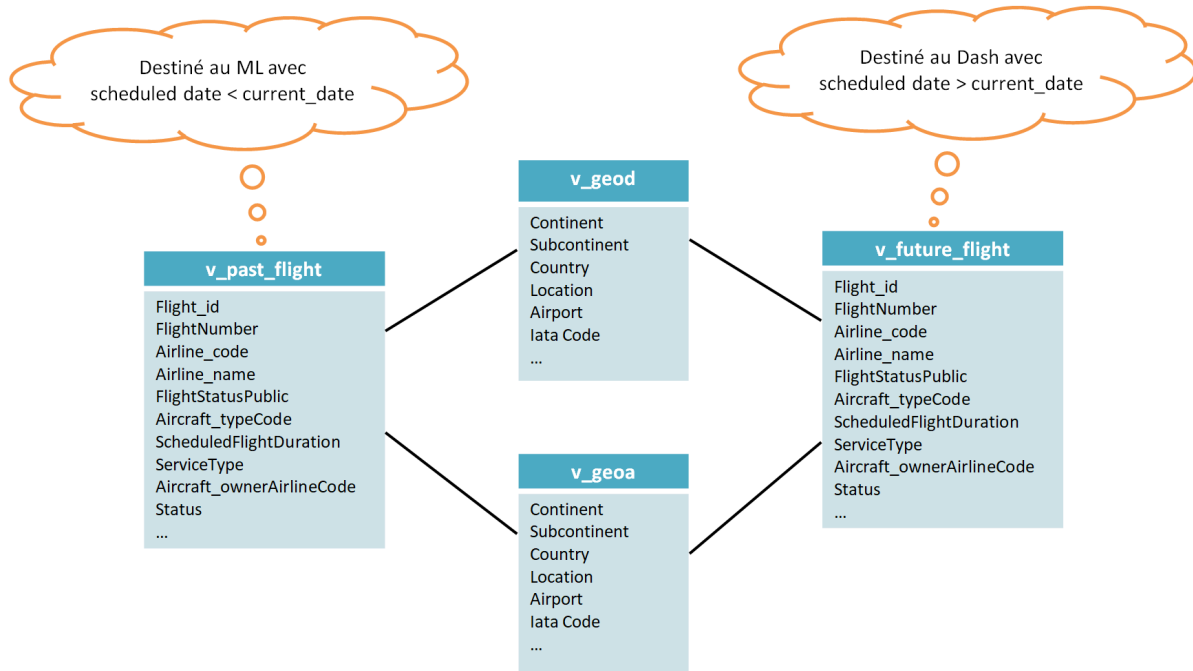
DROP TABLE IF EXISTS arrival_past CASCADE;
CREATE TABLE arrival_past (
arrival_id serial PRIMARY KEY,
flight_id varchar(50) UNIQUE NOT NULL,
flightLegs_arrInfo_airport_code varchar(3) NOT NULL,
flightLegs_arrInfo_airport_places_arrivalPositionTerminal varchar(50) NULL,
flightLegs_arrInfo_times_scheduled_date date NULL,
flightLegs_arrInfo_times_scheduled_time time NULL,
flightLegs_arrInfo_times_scheduled_year int NULL,
flightLegs_arrInfo_times_scheduled_month int NULL,
flightLegs_arrInfo_times_scheduled_day int NULL,
flightLegs_arrInfo_times_scheduled_hour int NULL,
flightLegs_arrInfo_times_scheduled_minute int NULL,
flightLegs_arrInfo_times_scheduled_timezone varchar(6) NULL,
flightLegs_arrInfo_times_number_week int NULL,
FOREIGN KEY (flight_id) REFERENCES flight_past(flight_id),
FOREIGN KEY (flightLegs_arrInfo_airport_code) REFERENCES Airport(lata_Code)
);

```

Création de vues

La création de vue a été nécessaire dans le cadre du projet dû à des problèmes de performances sur les machines virtuelles.

Ces vues ont permis de résoudre les problèmes de performance remplaçant des requêtes à plusieurs jointures et également l'interaction avec le machine learning pour les données du passé afin d'entraîner les modèles et les données futures pour l'interface utilisateur Dash permettant de sélectionner un vol futur et avoir sa prédiction sur le retard.



Requête fonctionnelle sur les vues

```
DROP VIEW IF EXISTS v_geod;
CREATE VIEW v_geod AS
select Continent.Continent_ID as flightLegs_deplInfo_airport_Continent_ID,
Continent.Continent_Name as flightLegs_deplInfo_airport_Continent_Name,
Subcontinent.Subcontinent_ID as flightLegs_deplInfo_airport_Subcontinent_ID,
Subcontinent.Subcontinent_Name as flightLegs_deplInfo_airport_Subcontinent_Name,
Country.Country_ID as flightLegs_deplInfo_airport_Country_ID,
Country.Country_Code as flightLegs_deplInfo_airport_Country_Code,
Country.Country_Name as flightLegs_deplInfo_airport_Country_Name,
Location.Location_ID as flightLegs_deplInfo_airport_Location_ID,
Location.Location_name as flightLegs_deplInfo_airport_Location_name,
Airport.Airport_ID as flightLegs_deplInfo_airport_Airport_ID,
Airport.Airport_Name as flightLegs_deplInfo_airport_Airport_Name,
Airport.Iata_Code as flightLegs_deplInfo_airport_Iata_Code,
Airport.Icao_Code as flightLegs_deplInfo_airport_Icao_Code,
Airport.Latitude as flightLegs_deplInfo_airport_Latitude,
Airport.Longitude as flightLegs_deplInfo_airport_Longitude
from Continent Continent
INNER JOIN Subcontinent ON Continent.Continent_ID = Subcontinent.Continent_ID
INNER JOIN Country ON Country.Subcontinent_ID = Subcontinent.Subcontinent_ID
INNER JOIN Location ON Location.Country_ID = Country.Country_ID
INNER JOIN Airport ON Airport.Location_ID = Location.Location_ID;
```

```
DROP VIEW IF EXISTS v_geoa;
CREATE VIEW v_geoa AS
select Continent.Continent_ID as flightLegs_arrInfo_airport_Continent_ID,
Continent.Continent_Name as flightLegs_arrInfo_airport_Continent_Name,
Subcontinent.Subcontinent_ID as flightLegs_arrInfo_airport_Subcontinent_ID,
Subcontinent.Subcontinent_Name as flightLegs_arrInfo_airport_Subcontinent_Name,
Country.Country_ID as flightLegs_arrInfo_airport_Country_ID,
Country.Country_Code as flightLegs_arrInfo_airport_Country_Code,
Country.Country_Name as flightLegs_arrInfo_airport_Country_Name,
Location.Location_ID as flightLegs_arrInfo_airport_Location_ID,
Location.Location_name as flightLegs_arrInfo_airport_Location_name,
Airport.Airport_ID as flightLegs_arrInfo_airport_Airport_ID,
Airport.Airport_Name as flightLegs_arrInfo_airport_Airport_Name,
Airport.Iata_Code as flightLegs_arrInfo_airport_Iata_Code,
Airport.Icao_Code as flightLegs_arrInfo_airport_Icao_Code,
Airport.Latitude as flightLegs_arrInfo_airport_Latitude,
Airport.Longitude as flightLegs_arrInfo_airport_Longitude
from Continent Continent
INNER JOIN Subcontinent ON Continent.Continent_ID = Subcontinent.Continent_ID
INNER JOIN Country ON Country.Subcontinent_ID = Subcontinent.Subcontinent_ID
INNER JOIN Location ON Location.Country_ID = Country.Country_ID
INNER JOIN Airport ON Airport.Location_ID = Location.Location_ID;
```

```
DROP VIEW IF EXISTS v_past_flight;
CREATE VIEW v_past_flight AS
select
flight_past.flight_id,
```

```

flight_past.flightNumber,
flight_past.airline_code,
flight_past.airline_name,
flight_past.flightStatusPublic,
flight_past.flightLegs_aircraft_typeCode,
flight_past.flightLegs_scheduledFlightDuration,
flight_past.flightLegs_serviceType,
flight_past.flightLegs_aircraft_ownerAirlineCode,
flight_past.flightLegs_status,
flight_past.delay_status,
flight_past.flightLegs_serviceTypeName,
flight_past.flightLegs_publishedStatus,
flight_past.flightLegs_legStatusPublic,
flight_past.flightLegs_statusName,
delay.flightLegs_irregularity_delayDuration,
delay.flightLegs_irregularity_delayduration_total,
delay.flightLegs_irregularity_delayInfo_delayReasonPublicLong,
delay.flightLegs_irregularity_delayInfo_delayReasonPublicShort,
departure_past.flightLegs_deplInfo_airport_code,
departure_past.flightLegs_deplInfo_airport_places_depPosTerm_boardingTerminal,
departure_past.flightLegs_deplInfo_airport_places_depPosTerm_gateNumber,
departure_past.flightLegs_deplInfo_times_scheduled_date,
departure_past.flightLegs_deplInfo_times_scheduled_time,
departure_past.flightLegs_deplInfo_times_scheduled_year,
departure_past.flightLegs_deplInfo_times_scheduled_month,
departure_past.flightLegs_deplInfo_times_scheduled_day ,
departure_past.flightLegs_deplInfo_times_scheduled_hour,
departure_past.flightLegs_deplInfo_times_scheduled_minute,
departure_past.flightLegs_deplInfo_times_scheduled_timezone,
departure_past.flightLegs_deplInfo_times_number_week,
arrival_past.flightLegs_arrInfo_airport_code,
arrival_past.flightLegs_arrInfo_airport_places_arrivalPositionTerminal,
arrival_past.flightLegs_arrInfo_times_scheduled_date,
arrival_past.flightLegs_arrInfo_times_scheduled_time,
arrival_past.flightLegs_arrInfo_times_scheduled_year,
arrival_past.flightLegs_arrInfo_times_scheduled_month,
arrival_past.flightLegs_arrInfo_times_scheduled_day,
arrival_past.flightLegs_arrInfo_times_scheduled_hour,
arrival_past.flightLegs_arrInfo_times_scheduled_minute,
arrival_past.flightLegs_arrInfo_times_scheduled_timezone,
arrival_past.flightLegs_arrInfo_times_number_week
from flight_past flight_past
    INNER JOIN delay ON delay.flight_id = flight_past.flight_id
    INNER JOIN departure_past ON departure_past.flight_id = flight_past.flight_id
    INNER JOIN arrival_past ON arrival_past.flight_id = flight_past.flight_id
where departure_past.flightLegs_deplInfo_times_scheduled_date < current_date;

```

```

DROP VIEW IF EXISTS v_future_flight;
CREATE VIEW v_future_flight AS
select
flight_future.flight_id,
flight_future.flightNumber,

```

```

flight_future.airline_code,
flight_future.airline_name,
flight_future.flightStatusPublic,
flight_future.flightLegs_aircraft_typeCode,
flight_future.flightLegs_scheduledFlightDuration,
flight_future.flightLegs_serviceType,
flight_future.flightLegs_aircraft_ownerAirlineCode,
flight_future.flightLegs_status,
flight_future.flightLegs_serviceTypeName,
flight_future.flightLegs_publishedStatus,
flight_future.flightLegs_legStatusPublic,
flight_future.flightLegs_statusName,
departure_future.flightLegs_deplInfo_airport_code,
departure_future.flightLegs_deplInfo_airport_places_depPosTerm_boardingTerminal,
departure_future.flightLegs_deplInfo_airport_places_depPosTerm_gateNumber,
departure_future.flightLegs_deplInfo_times_scheduled_date,
departure_future.flightLegs_deplInfo_times_scheduled_time,
departure_future.flightLegs_deplInfo_times_scheduled_year,
departure_future.flightLegs_deplInfo_times_scheduled_month,
departure_future.flightLegs_deplInfo_times_scheduled_day ,
departure_future.flightLegs_deplInfo_times_scheduled_hour,
departure_future.flightLegs_deplInfo_times_scheduled_minute,
departure_future.flightLegs_deplInfo_times_scheduled_timezone,
departure_future.flightLegs_deplInfo_times_number_week,
arrival_future.flightLegs_arrInfo_airport_code,
arrival_future.flightLegs_arrInfo_airport_places_arrivalPositionTerminal,
arrival_future.flightLegs_arrInfo_times_scheduled_date,
arrival_future.flightLegs_arrInfo_times_scheduled_time,
arrival_future.flightLegs_arrInfo_times_scheduled_year,
arrival_future.flightLegs_arrInfo_times_scheduled_month,
arrival_future.flightLegs_arrInfo_times_scheduled_day,
arrival_future.flightLegs_arrInfo_times_scheduled_hour,
arrival_future.flightLegs_arrInfo_times_scheduled_minute,
arrival_future.flightLegs_arrInfo_times_scheduled_timezone,
arrival_future.flightLegs_arrInfo_times_number_week
from flight_future
      INNER JOIN departure_future ON departure_future.flight_id = flight_future.flight_id
      INNER JOIN arrival_future ON arrival_future.flight_id = flight_future.flight_id
where departure_future.flightLegs_deplInfo_times_scheduled_date > current_date;

```

```

select v_past_flight.flight_id,
v_geod.flightLegs_deplInfo_airport_Continent_Name,
v_geod.flightLegs_deplInfo_airport_Subcontinent_Name,
v_geod.flightLegs_deplInfo_airport_Country_Name,
v_geod.flightLegs_deplInfo_airport_Location_name,
v_geod.flightLegs_deplInfo_airport_Airport_Name,
v_geod.flightLegs_deplInfo_airport_Iata_Code,
flightLegs_deplInfo_times_scheduled_date,
flightLegs_deplInfo_times_scheduled_time,
flightLegs_deplInfo_airport_places_depPosTerm_boardingTerminal,
flightLegs_deplInfo_airport_places_depPosTerm_gateNumber
from v_past_flight v_past_flight
inner join v_geod v_geod on v_geod.flightLegs_deplInfo_airport_iata_code =
v_past_flight.flightLegs_deplInfo_airport_code
where v_geod.flightLegs_deplInfo_airport_Location_name = 'Ile-de-France';

```

Données Messages Notifications											
Affichage des lignes : 1 à 1000 Page No: 1 de 176											
	flight_id character varying (50)	flightlegs_d character varying (10)	flightlegs_depinfo_ai character varying (10)	flightlegs_de character varying (10)	flightlegs_depinfo character varying (100)	flightlegs_depinfo_airport character varying (100)	flightlegs character	flightlegs_depinfo date	flightlegs_dep time without t	flightleg character	flightleg character
17	20250515+AF+1558CDGNCL	Europe	Western Europe	France	Ile-de-France	Paris Charles de Gaulle Airport	CDG	2025-05-15	21:00:00	2E	K23
18	20250515+AF+1580CDGLHR	Europe	Western Europe	France	Ile-de-France	Paris Charles de Gaulle Airport	CDG	2025-05-15	10:00:00	2E	K57
19	20250515+AF+1596CDGCMV	Europe	Western Europe	France	Ile-de-France	Paris Charles de Gaulle Airport	CDG	2025-05-15	12:40:00	2E	L29
20	20250515+AF+1642CDGGGA	Europe	Western Europe	France	Ile-de-France	Paris Charles de Gaulle Airport	CDG	2025-05-15	08:30:00	2F	F47
21	20250515+TO+3888ORYBRI	Europe	Western Europe	France	Ile-de-France	Aéroport de Paris-Orly	ORY	2025-05-15	17:10:00	3	E09
22	20250515+TO+4370ORYPRG	Europe	Western Europe	France	Ile-de-France	Aéroport de Paris-Orly	ORY	2025-05-15	19:50:00	3	E04
23	20250515+TO+4762ORYIBZ	Europe	Western Europe	France	Ile-de-France	Aéroport de Paris-Orly	ORY	2025-05-15	14:20:00	3	E14
24	20250515+TO+8152ORYMIR	Europe	Western Europe	France	Ile-de-France	Aéroport de Paris-Orly	ORY	2025-05-15	20:50:00	3	D15
Nombre total de lignes : 175812		Requête terminée 00:00:03.982							CRLF Lgn 15, Col		

La base de données relationnelle mise en place permet de faire des requêtes pour récupérer les informations à partir du continent jusqu'à la porte d'embarquement du vol (si elle est disponible).



**La Bdd stockant pour l'heure les objets "flights"
récupérés sur l'API AFKLM
d'Air France**



Données utiles de la source

Dans le cadre de notre projet de groupe, dont l'objectif est de prévoir le retard des vols à partir de données réelles issues de l'API Air France/KLM, nous avons déterminé les champs pertinents d'un fichier JSON.

L'analyse a été réalisée à partir d'un fichier JSON complet provenant de l'API Flight Status d'Air France/KLM.

Objectif de la sélection

L'objectif de cette étape était de sélectionner uniquement les variables essentielles au projet, afin d'éviter la surcharge d'informations inutiles et de ne conserver que celles :

- Nécessaires à la construction du pipeline de données (identification, extraction, stockage, transformation)
- Pertinentes pour la phase de modélisation prédictive (machine learning sur le retard des vols)

L'approche suivie repose donc sur une logique de pertinence métier et technique, en privilégiant les variables qui ont un impact direct ou indirect sur le retard d'un vol.

Structure générale des données renvoyées par l'API Air France KLM

Le fichier renvoyé par l'API contient une structure hiérarchique composée de plusieurs niveaux :

operationalFlights	Informations générales du vol (numéro, compagnie, type d'appareil, statut, horaires, etc.)
flightLegs	Détails des étapes du vol (départ, arrivée, statuts internes)
departureInformation arrivalInformation	Détails sur les aéroports, horaires et terminaux

L'ensemble des champs a été analysé afin d'identifier ceux réellement exploitables pour la suite du projet.

Données retenues

Les données retenues ont été regroupées en quatre catégories principales :

Identification du vol

Ces variables permettent de reconnaître et relier chaque vol à sa compagnie, son type d'appareil et son service.

Informations horaires

Ce sont des variables fondamentales pour calculer les retards, comparer les horaires planifiés, estimés et réels.

Localisation et aéroports

Il s'agit d'informations de départ et d'arrivée (codes, pays, terminaux) servant à croiser les données externes.

Statut et indicateurs de vol

Ces variables décrivent l'état du vol et les éventuels retards.

Compléments à prévoir

Certaines informations contextuelles ne sont pas directement présentes dans l'API Air France/KLM, mais peuvent être enrichies à partir d'autres sources.

Récapitulatif

Cette phase de sélection a permis d'isoler un ensemble restreint de champs à forte valeur ajoutée pour le projet, garantissant :

- Une base de données structurée
- Une meilleure lisibilité du pipeline
- Une meilleure qualité des features pour la modélisation des retards de vols

Ce travail servira directement à la construction du schéma de données, la transformation, ainsi qu'à la définition des variables du modèle prédictif final.

Catégorie : Identification du vol

Champ	Description	Type	Pertinence
flightNumber	Numéro de vol (unique par compagnie et jour)	VARCHAR(10) not null	Clé d'identification du vol
FlightLegs.aircraft.owner AirlineCode	Code de la compagnie (ex: AF, KL)	CHAR(2) not null	Nécessaire pour regrouper par compagnie

FlightLegs.aircraft.typeCode	Type d'avion utilisé	VARCHAR (10) null	Peut influencer les retards selon modèle
FlightLegs.serviceType	Type de service (passager, cargo, mixte)	VARCHAR (15) null	Information contextuelle secondaire

Catégorie : Informations horaires

Champ	Description	Type	Pertinence
FlightLegs.departureInformation.times.scheduled	Heure prévue de départ	DATETIME not null	Base pour calcul du retard
FlightLegs.departureInformation.times.latestPublished	Heure publiée la plus récente	DATETIME not null	Indice du retard en temps réel
FlightLegs.departureInformation.times.actual	Heure réelle de départ	DATETIME null	Référence finale pour calcul du retard réel
FlightLegs.arrivalInformation.times.scheduled	Heure prévue d'arrivée	DATETIME not null	Pour comparaison retard à l'arrivée
FlightLegs.arrivalInformation.times.estimated.value	Heure estimée d'arrivée	DATETIME not null	Donnée prédictive utile
FlightLegs.arrivalInformation.times.actual	Heure réelle d'arrivée	DATETIME null	Cible de validation du modèle
FlightLegs.scheduledFlightDuration	Durée planifiée du vol en minutes	INT null	Facteur secondaire du modèle (minutes)

Catégorie : Localisation / Aéroports

Champ	Description	Type	Pertinence
-------	-------------	------	------------

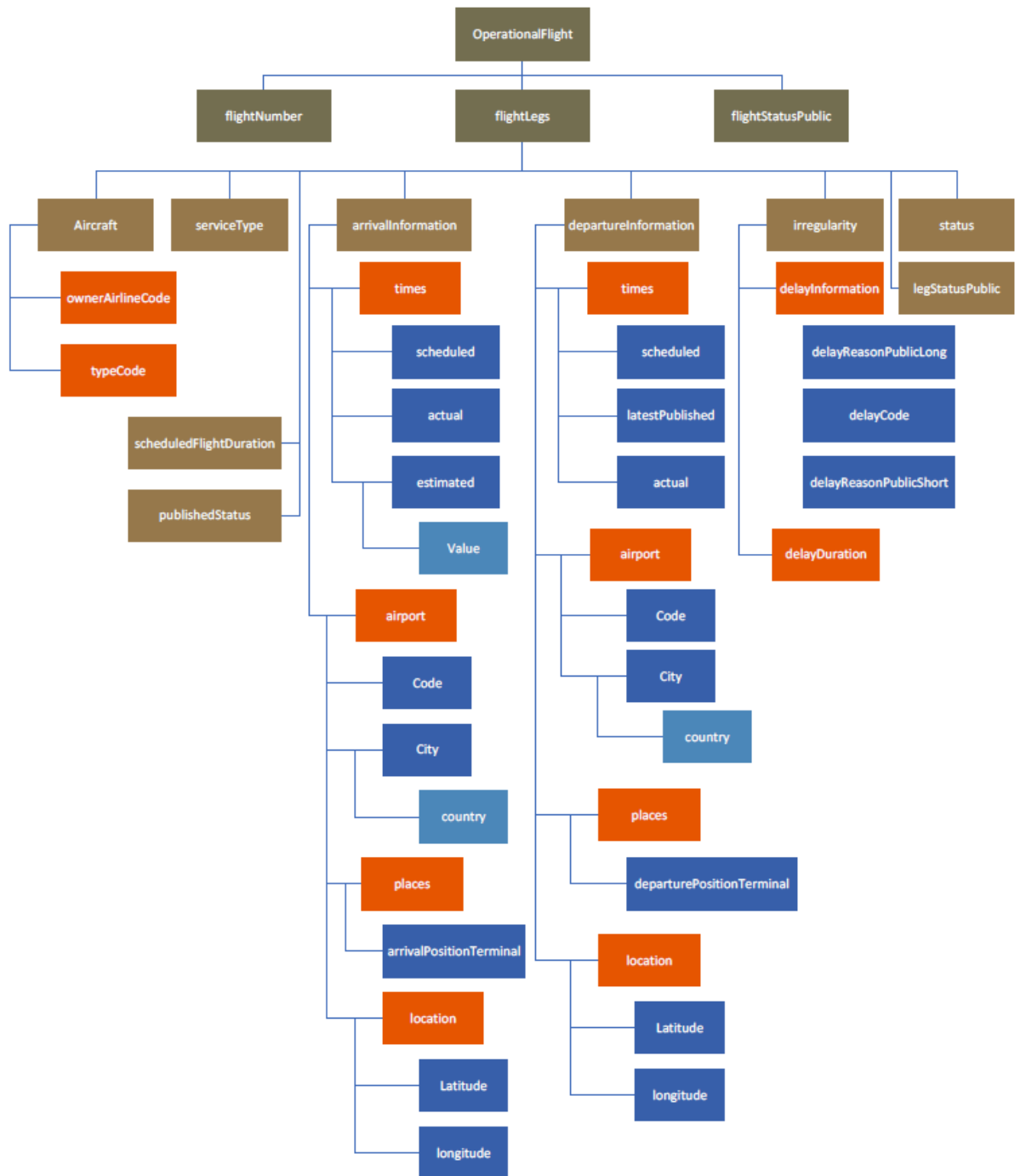
FlightLegs.departureInformation.airport.code	Aéroport de départ (IATA)	CHAR(3) not null	Permet d'analyser le hub de départ
FlightLegs.departureInformation.airport.city.country	Pays de départ	CHAR(2) not null	Grouper les vols par région ou fuseau horaire
FlightLegs.arrivalInformation.airport.code	Aéroport d'arrivée (IATA)	CHAR(3) not null	Permet d'analyser les destinations
FlightLegs.arrivalInformation.airport.city.country	Pays d'arrivée	CHAR(2) not null	Distinguer vols domestiques et internationaux
FlightLegs.departureInformation.airport.places .departurePositionTerminal	Terminal de départ	VARCHAR(5) null	Indicateur logistique
FlightLegs.arrivalInformation.airport.places .arrivalPositionTerminal	Terminal d'arrivée	VARCHAR(5) null	Comprendre les retards liés aux terminaux
FlightLegs.departureInformation.airport.location.latitude	Latitude de l'aéroport de départ	FLOAT NULL	Position géographique du départ
FlightLegs.departureInformation.airport.location.longitude	Longitude de l'aéroport de départ	FLOAT NULL	Complète la latitude
FlightLegs.arrivalInformation.airport.location.latitude	Latitude de l'aéroport d'arrivée	FLOAT NULL	Mesure la distance départ/arrivée

FlightLegs.arrivalInformation.airport.location.longitude	Longitude de l'aéroport d'arrivée	FLOAT NULL	Calcul exact de la distance de vol
---	-----------------------------------	---------------	------------------------------------

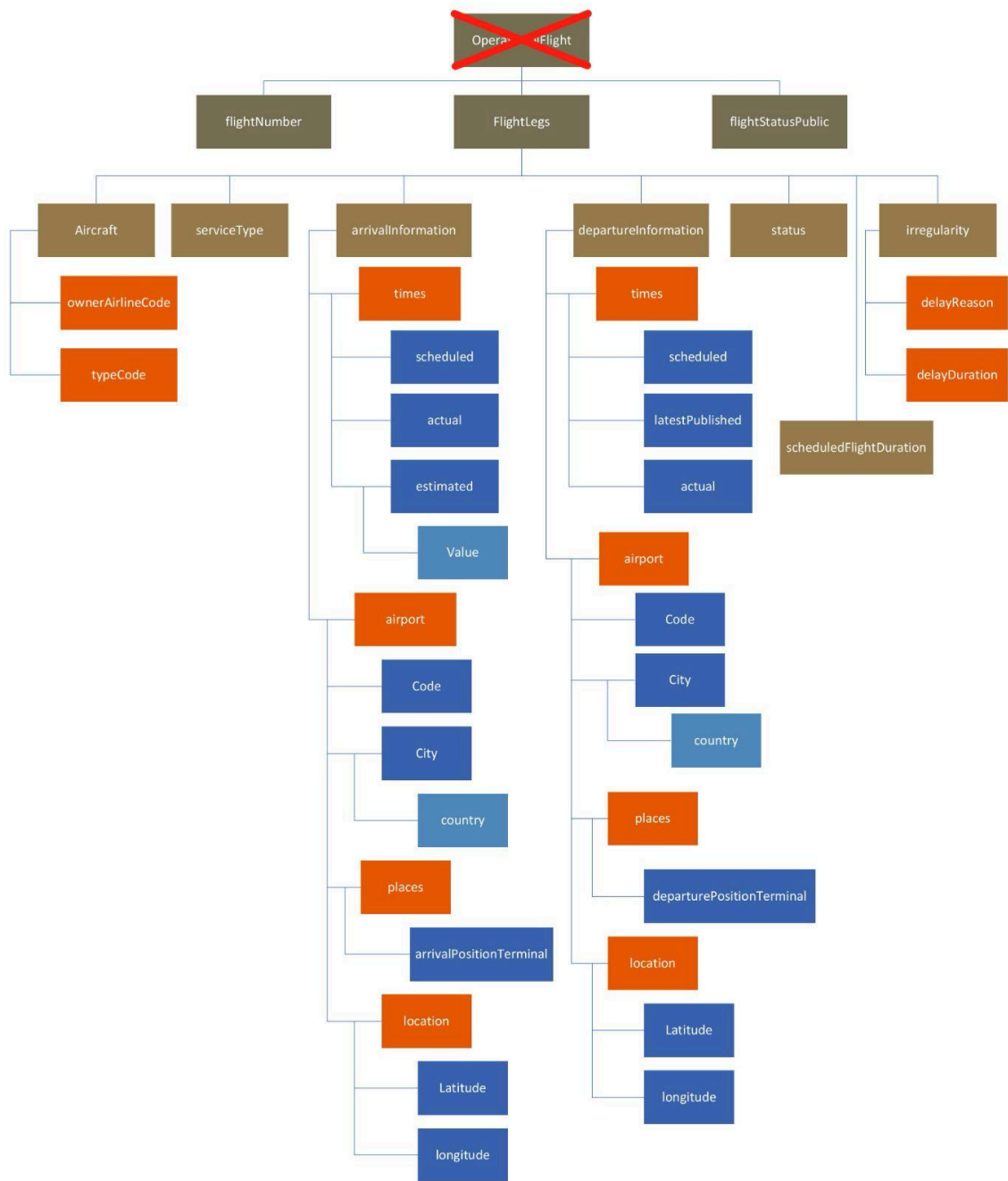
Catégorie : Statut du vol et indicateurs

Champ	Description	Type	Pertinence
flightStatusPublic	Statut public du vol (ON_TIME, DELAYED, CANCELLED, etc.)	VARCHAR(30) not null	Référence principale (vol ponctuel, en retard, annulé)
FlightLegs.status	Code interne du statut	CHAR(1) not null	Filtre ou classe les vols selon leur état opérationnel
FlightLegs.irregularity.delayReason	Cause du retard (si renseignée)	VARCHAR(50) null	Indication sur les causes possibles de retard
FlightLegs.irregularity.delayDuration	Retard cumulé en minutes	INT null	Variable cible principale pour la prédiction du retard

Arborescence des données retenue



Schema de l'objet stocké



Note: Les objets flights étaient dans des opérations flights. Cependant ce sont bien des vols que l'on souhaite récupérer, nous avons donc extrait les vols de opération flight pour les stocker dans une collection flight.

Voici un flight telle qu'il est stocker en BDD:

```
{
  "_id": "6900d152be9ad5e05baf37d0",
  "flightNumber": 1219,
  "flightScheduleDate": "2025-06-04",
  "id": "20250604+SK+1219",
  "route": [
    "CPH",
    "AAL"
  ],
  "airline": {
    "code": "SK",
    "name": "SCANDINAVIAN AIRLINES SYSTEM (SAS)"
  },
  "codeShareRelations": [
    {
      "marketingFlightNumber": 4934,
      "code": "F",
      "type": "PASSENGER",
      "airline": {
        "code": "AF",
        "name": "AIR FRANCE"
      }
    },
    {
      "marketingFlightNumber": 8412,
      "airline": {
        "code": "KL",
        "name": "KLM ROYAL DUTCH AIRLINES"
      }
    }
  ],
  "flightStatusPublic": "ARRIVED",
  "flightStatusPublicLangTransl": "Arrived",
}
```

```
"flightLegs": [  
  {  
    "publishedStatus": "ARRIVED",  
    "departureInformation": {  
      "airport": {  
        "code": "CPH",  
        "name": "KASTRUP AIRPORT",  
        "nameLangTranl": "Copenhagen Airport",  
        "city": {  
          "code": "CPH",  
          "name": "COPENHAGEN",  
          "nameLangTranl": "Copenhagen",  
          "country": {  
            "areaCode": "E",  
            "code": "DK",  
            "name": "DENMARK",  
            "nameLangTranl": "Denmark",  
            "euroCountry": "N",  
            "euCountry": "Y"  
          }  
        },  
      },  
      "location": {  
        "latitude": 55.6081,  
        "longitude": 12.6333  
      },  
      "places": {  
        "gateNumber": [  
          "B8"  
        ],  
        "terminalCode": "3",  
        "boardingTerminal": "3",  
        "checkInZone": [],  
        "paxDepartureGate": []  
      }  
    },  
    "times": {  
      "actual": "2025-06-04T19:49:00.000+02:00",
```



```
    "scheduled": "2025-06-04T19:55:00.000+02:00",
    "latestPublished": "2025-06-04T19:49:00.000+02:00",
    "actualTakeOffTime": "2025-06-04T19:58:00.000+02:00"
  }
},
"arrivalInformation": {
  "airport": {
    "code": "AAL",
    "name": "AALBORG AIRPORT",
    "nameLangTranl": "Aalborg",
    "city": {
      "code": "AAL",
      "name": "AALBORG",
      "nameLangTranl": "Aalborg",
      "country": {
        "areaCode": "E",
        "code": "DK",
        "name": "DENMARK",
        "nameLangTranl": "Denmark",
        "euroCountry": "N",
        "euCountry": "Y"
      }
    }
  },
  "location": {
    "latitude": 57.0931,
    "longitude": 9.85
  },
  "places": {
    "gateNumber": [],
    "baggageBelt": [
      "2"
    ],
    "arrivalHall": []
  }
},
"times": {
  "actual": "2025-06-04T20:32:00.000+02:00",
```

```
"scheduled": "2025-06-04T20:40:00.000+02:00",
"latestPublished": "2025-06-04T20:32:00.000+02:00",
"estimated": {
  "value": "2025-06-04T20:32:00.000+02:00"
},
"actualTouchDownTime": "2025-06-04T20:26:00.000+02:00"
},
"legStatusPublic": "ARRIVED",
"legStatusPublicLangTransl": "Arrived",
"passengerCustomsStatus": "SCHENGEN",
"serviceType": "J",
"serviceName": "Normal Service",
"restricted": false,
"scheduledFlightDuration": "PT45M",
"departureDateTimeDifference": "PT-6M",
"arrivalDateTimeDifference": "PT-8M",
"timeToArrival": "PT0S",
"completionPercentage": "100",
"timeZoneDifference": "+0000",
"aircraft": {
  "registration": "SEROC",
  "typeCode": "32N",
  "typeName": "AIRBUS A320NEO",
  "ownerAirlineCode": "SK",
  "ownerAirlineName": "SCANDINAVIAN AIRLINES SYSTEM (SAS)",
  "physicalPaxConfiguration": "C024M156",
  "physicalFreightConfiguration": "PP000LL000",
  "operationalConfiguration": "CDZJYSBPAEMHQVWURKNLTOG",
  "cockpitCrewEmployer": "SK",
  "cabinCrewEmployer": "SK"
},
"irregularity": {
  "cancelled": "N",
  "delayCode": [],
  "delayReason": [],
  "delayDuration": [],
```

```
"delayReasonCodePublic": [],  
"delayReasonPublicLangTransl": [],  
"delayReasonPublic": [],  
"delayInformation": []  
}  
}  
]  
}
```

Les Outils d'interactions avec la BDD

1 readme

1 .sh qui permet : Démarrage container docker + BDD + Import dump.archive dans la BDD

2 .sh permettant d'importer les json de l'api Airfrance (Simulation de situation d'entreprise : Un outil pour import pour les personnes internes, et un outil simulant un client externe qui doit passer à terme par des appels api pour insérer de la data dans la base. Pour le moment, cet outil appelle les use cases, mais à terme il pourrait appeler les routes api qui ne sont pas dev à ce stade du projet).

1 Route api pour sortir un dump en .archive

1 Ligne de commande fourni dans le readme pour exporter un dump .archive

1 Ligne de commande fourni dans le readme pour importer un dump .archive

1 Route api pour sortir une version tabulaire .csv de la data.

1 .sh qui fait la même chose

1 Api (routes visibles sur l'image ci dessous, cependant, c'est une première ébauche, il n'a pas encore été décidé si on fait du rest, si on va au bout de cet api, si elle reste seulement un échantillon pour illustrer ce qui pourrait être une bonne pratique avec plus de temps.)

AirlinesApi 0.1.0 OAS 3.1

/openapi.json

default

GET

/flights/{id} Read Flight

GET

/all/flights/ Read Flight

GET

/count_documents_by_collection/ Count Doc

GET

/dump/all Get Full Dump

Schemas

HTTPValidationError > Found all object

L'intérêt de l'Api: Permettre l'interaction du client avec la BDD dans le respect d'un contrat garantissant la sécurité de la BDD et facilitant l'interaction du client avec la BDD. Il ne peut y avoir d'interaction entre la BDD et le Client qu'en passant par l'API, et l'API garantit que le contrat est respecté.

ARCHITECTURE DU CODE

```
bdd/MongoDb/
├── mongo_db_interaction/
│   ├── main.py
│   ├── SERIALIZER/
│   ├── USE_CASES/
│   ├── SERVICES/
│   ├── REPOSITOIRES/
│   └── DB_CONTEXT/
├── data_dump/
├── docker-compose.yml
├── .env
└── envstruct
```

Il s'agit d'un échantillon de ce que pourrait être une architecture inspirée de l'inversion de dépendance et basée sur la single responsibility.

Il en reste que les use cases sont agnostiques de la BDD. Cependant, prenons l'exemple du use_case insert_flight_uc:

Comme on peut le voir, il est agnostique de la BDD... Qu'est ce qu'il fait?

Il insert un flight, et vérifie si à l'endroit où il insert le flight, (à cette échelle on ignore si c'est dans une Bdd et quelle BDD), si l'id de ce flight est déjà présent.

On voit dans le code joint ci-après, qu'il appelle directement le get_by_id du repository, plutôt qu'utiliser le use case get_by_id développé pour l'api. En fait, à terme, il devrait faire appel au use case get_by_id_flight.

Pourquoi? Parce que le use case get_by_id_flight est aussi censé faire appel à un service qui lui va vérifier si l'id est conforme au format d'id attendu. Ce sont des choses non faites pour le moment, mais des améliorations possibles.

De plus, avec ce type de développement, on peut lire le use case, comprendre ce qu'il fait, sans aller sur la couche du dessous.

Donc si on venait à changer de BDD, sans changer de logique métier, il n'y aurait que les Repositories à modifier. Car on insérerait toujours un flight en appelant le get_by_id, que ce soit pour le rentrer dans une bdd SQL, ou noSQL... Par exemple.

```
from ..REPOSITORIES.flights import insert_one, get_by_id

def insert_flight(flight):
    id = flight['id']

    if is_flight_exit(id) == False:
        insert_one(flight)

def is_flight_exit(id):
    flight_id = get_by_id(id)
    if flight_id == None:
        print(id)
        return False

    else:
        print(id + " Already exist")
        return True
```

I

CONCLUSION:

Les fondations de la BDD MongoDB sont posées, et des outils d'interaction pour les clients de la BDD sont disponibles et utilisables.

Étape 3 - Consommation de la donnée

Etude des données tabulaires

Contexte

Dans le cadre du projet de prédiction des retards de vols, les données issues de l'API Air France-KLM ont été collectées au format JSON, transformées en CSV puis analysées.

L'objectif de cette étape est de :

- Identifier les champs réellement exploitables pour l'entraînement du modèle prédictif
- Mesurer la proportion de valeurs manquantes dans chaque variable
- Déterminer quelles variables doivent être conservées, ignorées ou réservées pour de l'analyse descriptive

Structuration des données

Les variables ont été réparties en quatre catégories selon leur rôle dans la modélisation :

- Variables directement utilisées par le modèle pour apprendre à prédire le retard (explicatives)
- Variables complémentaires servant à interpréter ou illustrer les résultats (descriptives)
- Variable à prédire, représentant l'état final du vol (cible)
- Variables exclues du jeu d'apprentissage pour cause de redondance, d'inutilité ou de complétude insuffisante (non retenues)

Variables explicatives

Champ	Taux de valeurs manquantes
flightLegs-status	16.99
flightLegs-scheduledFlightDuration	8.37
flightLegs-aircraft-ownerAirlineCode	6.04
flightLegs-serviceType	0.0
flightLegs-aircraft-typeCode	0.0

flightLegs-departureInformation-times-scheduled	0.0
flightLegs-arrivalInformation-times-scheduled	0.0
flightLegs-departureInformation-airport-code	0.0
flightLegs-arrivalInformation-airport-code	0.0
flightLegs-departureInformation-airport-city-country-code	0.0
flightLegs-arrivalInformation-airport-city-country-code	0.0
flightLegs-departureInformation-airport-city-country-areaCode	0.0
flightLegs-arrivalInformation-airport-city-country-areaCode	0.0
flightLegs-departureInformation-airport-location-latitude	0.0
flightLegs-departureInformation-airport-location-longitude	0.0
flightLegs-arrivalInformation-airport-location-latitude	0.0
flightLegs-arrivalInformation-airport-location-longitude	0.0
flightNumber	0.0

Variables descriptives

Champ	Taux de valeurs manquantes
flightLegs-irregularity-delayInformation-delayReasonPublicLong	88.73
flightLegs-irregularity-delayInformation-delayReasonPublicShort	88.73
flightLegs-departureInformation-airport-places-departurePositionTerminal-boardingTerminal	70.93
flightLegs-arrivalInformation-airport-places-arrivalPositionTerminal	59.42
flightLegs-departureInformation-airport-places-departurePositionTerminal-gateNumber	46.89
flightLegs-arrivalInformation-airport-city-country-name	0.0
flightLegs-departureInformation-airport-city-country-name	0.0

Variable cible

Champ	Taux de valeurs manquantes
FlightStatusPublic	0.0
flightLegs-irregularity-delayDuration	74.01

Variables non retenues

Champ	Taux de valeurs manquantes
flightLegs-irregularity-delayReason	100.0
flightLegs-departureInformation-times-actualTakeOffTime	43.11
flightLegs-departureInformation-times-actual	42.96
flightLegs-arrivalInformation-times-actual	39.06
flightLegs-arrivalInformation-times-actualTouchDownTime	38.96
flightLegs-arrivalInformation-times-latestPublished	0.27
flightLegs-arrivalInformation-times-estimated-value	0.27
flightLegs-departureInformation-times-latestPublished	0.26

Analyse des catégories

flightStatusPublic

flightStatusPublic	Taux
ARRIVED	0.909618
ON_TIME	0.063893
CANCELLED	0.022626

PARTIALLY_CANCELLED	0.003707
DELAYED_DEPARTURE	0.000142
DEPARTED	0.000007
NEW_DEPARTURE_TIME	0.000007

flightLegs-serviceType

Type	Correspondance du type	Taux
J	Normal Service	0.947830
U	Service operated by Surface Vehicle	0.043422
C	Passenger Only	0.005516
G	Normal Service	0.003185
S	Shuttle Mode	0.000020
L	Passenger and Cargo/ Mail	0.000020
O	Migrants/ Immigrants	0.000007

flightLegs-status

Statut	Libellé du statut	Taux
S	Scheduled	0.928392
C	Cancelled	0.070408
N	New	0.000653
G	Ground return	0.000237
T	Return to Ramp	0.000220
A	Air return	0.000049
R	Return from Airborn	0.000041

Exploration des données

Contexte

L'exploration des données a pour but de nettoyer et préparer le jeu de données, c'est-à-dire d'enlever les champs ou valeurs non pertinents pour la modélisation et d'identifier les vols réellement en retard ou anormaux. Le jeu de données étudié est issu d'un export CSV de plus de 700 000 enregistrements.

Analyse de la variable `flightStatusPublic`

<code>flightStatusPublic</code>	Taux (%)
ARRIVED	91,2516
ON_TIME	5,5515
CANCELLED	3,0631
PARTIALLY_CANCELLED	0,1323
DELAYED_DEPARTURE	0,0013
DEPARTED	0,0001
NEW_DEPARTURE_TIME	0,0001

Cette variable décrit l'état global d'un vol. La majorité des enregistrements concernent des vols arrivés (91,25 %), suivis des vols à l'heure (5,55 %) et annulés (3,06 %).

Les catégories `DEPARTED` et `NEW_DEPARTURE_TIME` sont considérées comme des valeurs extrêmes et incomplètes :

- o `DEPARTED` correspond à un vol parti sans qu'un statut d'arrivée ait été renseigné
- o `NEW_DEPARTURE_TIME` désigne une donnée administrative liée à la replanification des horaires

Ces deux catégories doivent être supprimées du jeu d'entraînement afin de ne pas perturber le modèle prédictif.

Analyse de la variable flightLegs_serviceType

Type	Correspondance du type	Taux
J	Normal Service (vol programmé)	98,6143
U	Service operated by Surface Vehicle	0,7257
C	Passenger Only	0,3500
G	Normal Service (vol non planifié)	0,2948
S	Shuttle Mode	0,0121
O	Migrants/ Immigrants	0,0025
L	Passenger and Cargo/ Mail	0,0005

Cette variable décrit le type d'opération du vol. La grande majorité des vols sont de type J (Normal Service) avec 98,61 %.

Le type G correspond à des vols additionnels non prévus dans le plan initial. Bien que le libellé soit identique à J, leur occurrence est très faible (0,29%). Leur similarité opérationnelle justifiera leur fusion afin de simplifier le modèle de prédiction.

Les types L et O, correspondant respectivement à Passenger & Cargo/Mail et Migrant/Immigrant, devront être supprimés. Ces catégories sont trop rares pour être significatives et doivent être écartées du jeu d'entraînement.

Analyse de la variable flightLegs_status

Statut	Libellé du statut	Taux
S	Scheduled	92,2567
C	Cancelled	7,6491
N	New	0,0474
T	Return to Ramp	0,0236
G	Ground return	0,0120
R	Return from Airborn	0,0079
A	Air return	0,0033

Cette variable décrit l'état du vol au niveau opérationnel. Les valeurs principales sont S pour les vols planifiés (92,25%) et C pour les vols annulés 7,65%).

Les valeurs N (New), G (Ground Return), T (Return to Ramp), A (Air Return) et R (Return from Airborne) représentent des cas particuliers liés à des anomalies ou retours d'urgence.

Le statut N étant un statut prévisionnel non validé, il est supprimé.

Les autres valeurs rares mais opérationnellement significatives sont conservées pour détecter des comportements anormaux lors de l'apprentissage.

Synthèse des irrégularités par flightStatusPublic

flightStatusPublic	Total vols	Vols avec irrégularité	% d'irrégularités
ARRIVED	694815	182386	26,25
CANCELLED	23323	1295	5,55
ON_TIME	42271	452	1,07
PARTIALLY_CANCELLED	1007	25	2,48
DELAYED_DEPARTURE	10	4	40
DEPARTED	1	0	0
NEW_DEPARTURE_TIME	1	0	0

Synthèse des irrégularités par flightLegs_status

flightLegs_status	Total vols	Vols avec irrégularité	% d'irrégularités
S	582540	182613	31,35
C	48299	1480	3,06
T	149	49	32,89
R	50	11	22
N	299	6	2,01
G	76	3	3,95
A	21	0	0

Synthèse des irrégularités par flightLegs_serviceType

flightLegs_serviceType	Total vols	Vols avec irrégularité	% d'irrégularités
J	750877	183375	24,42
G	2245	608	27,08
C	2665	129	4,84
U	5526	50	0,90
L	4	0	0
O	19	0	0
S	92	0	0

Top 10 des flightLegs_aircraft_ownerAirlineCode par taux d'irrégularités

ownerAirlineCode	Total vols	Vols avec irrégularité	% d'irrégularités
R6	2	2	100
T3	3314	2846	85,88
ZQ	6846	5406	78,97
KL	50755	38849	76,54
WA	46806	35256	75,32
ED	4	3	75
AF	75895	56531	74,49
JP	9	6	66,67
A5	20981	13808	65,81
XK	8018	4983	62,15

Top 10 des flightLegs_aircraft_typeCode par taux d'irrégularités

typeCode	Total vols	Vols avec irrégularité	% d'irrégularités
295	14496	12368	85,32
73J	4106	3078	74,96
318	4150	2810	67,71
E90	55593	31747	57,11
772	5945	3317	55,79
E70	7112	3835	53,92
223	36574	18750	51,27
77W	8359	4012	48
73W	9060	3920	43,27
E7W	25113	9973	39,71

L'exploration des données a permis :

- o De supprimer ou fusionner les valeurs extrêmes et incohérentes
- o D'identifier les variables explicatives fiables pour le modèle prédictif

Pipeline de Machine Learning avec scikit-learn

Objectif

- Depuis les données obtenues par l'API Air France - KLM, prédire l'occurrence de retard/annulation et la durée du retard le cas échéant sur les vols futurs en se basant sur les vols historiques



Jeu de données

- Toutes les données obtenues par le biais de l'API Air France KLM au cours du projet

Sélection des données

- Vols partant ou arrivant dans un des aéroports du Top 30 européen en termes de nombre de vols journaliers
- > 550'000 vols conservés

Feature engineering

- `flightLegs_scheduledFlightDuration = flightLegs_arrivalInformation_times_scheduled - flightLegs_departureInformation_times_scheduled`
- `flightLegs_Category`
 - 'LATE' si `'flightLegs_irregularity_delayDuration_total' > 0`
 - 'CANCELLED' si `flightLegs_legStatusPublic = 'CANCELLED'`
 - Sinon 'ONTIME'
- Saisonnalité
 - décembre - février → Hiver
 - mars - mai → Printemps
 - juin - août → Été
 - septembre - novembre → Automne
- Période journalière (pour arrivée/destination séparément)
 - 00:00:00 - 05:59:59 → Nuit
 - 00:60:00 - 11:59:59 → Matin
 - 00:12:00 - 17:59:59 → Après-midi
 - 00:18:00 - 23:59:59 → Soir
- Weekend (Samedi/Dimanche) (pour arrivée/destination séparément)

Feature preprocessing

- Données numériques
 - Pipeline([('imputer', SimpleImputer(strategy='median')),('scaler', StandardScaler())])
- Données catégorielles
 - Pipeline([('imputer', SimpleImputer(strategy='most_frequent'))
- One-hot encoding
 - ('onehot', OneHotEncoder(handle_unknown='ignore', drop='first', sparse_output=True)))

Statut du vol (classification)

Séparation jeux de données

- entraînement 80% > 450'000 vols
- test 20% > 110'000 vols

Type de données

- Valeur cible : flightLegs_Category
 - ONTIME
 - LATE
 - CANCELLED
- Valeurs explicatives
 - Catégorielles
 - 'flightLegs_aircraft_typeCode'
 - 'flightLegs_arrivalInformation_airport_code'
 - 'flightLegs_departureInformation_airport_code'
 - 'flightLegs_season'
 - 'flightLegs_arrivalInformation_times_scheduled_dayPeriod'
 - 'flightLegs_departureInformation_times_scheduled_dayPeriod'
 - Numériques
 - 'flightLegs_scheduledFlightDuration'

Hyperparameter tuning

- GridSearchCV (petit nombre de paramètres pour tester le pipeline)
- cv = 5
- scoring = accuracy

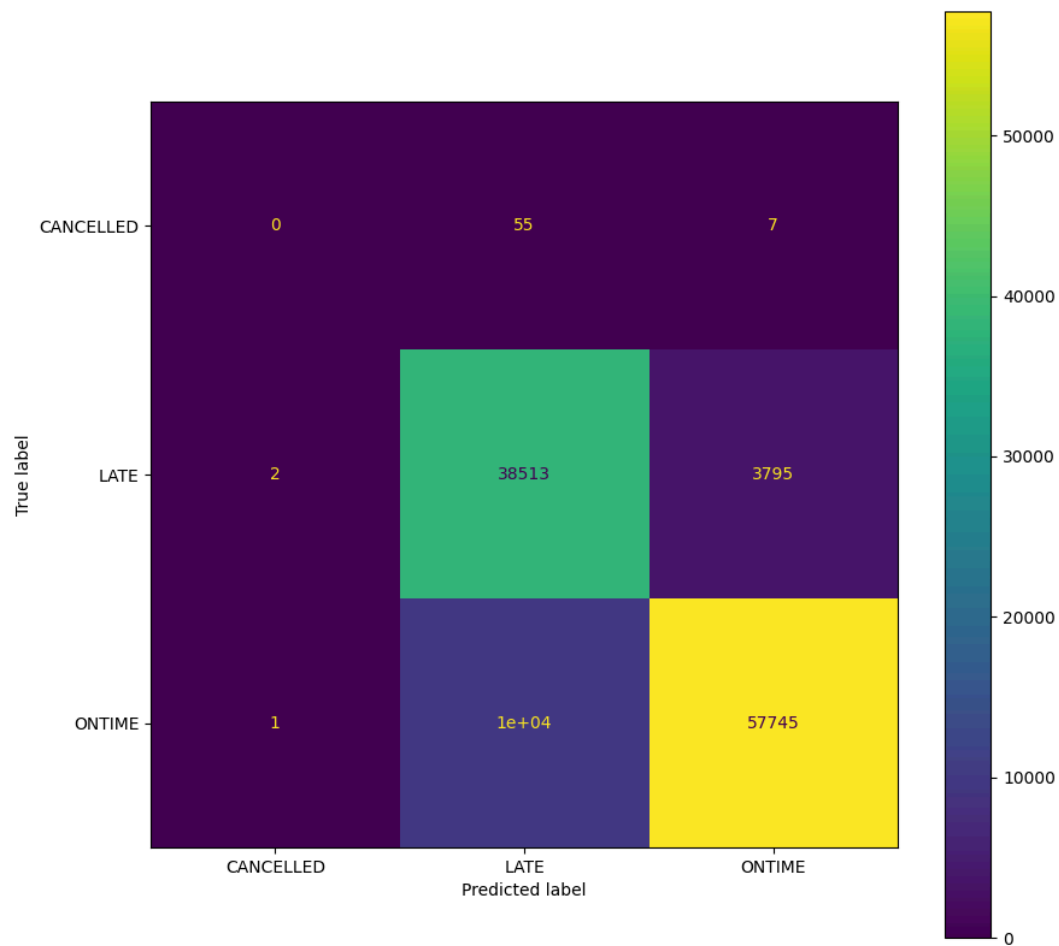
Algorithmes

- Logistic_OVO (One vs One)
 - best score: 0.86
- Logistic_OVR (One vs Rest)

- best score: 0.86
- DecisionTree
 - best score: 0.87
- RandomForest
 - best score: 0.87

Exemple de Matrice de confusion

Delay Status - RandomForest



Conclusion

Bonne précision (87%).

Durée du retard (classification)

Séparation jeux de données

- Vols avec flightLegs_irregularity_delayDuration_total > 0
- entraînement 80% > 169'000 vols
- test 20% > 42'000 vols

Type de données

- Valeur cible : flightLegs_irregularity_delayDuration_total
- Valeurs explicatives
 - Catégorielles
 - 'flightLegs_aircraft_typeCode'
 - 'flightLegs_arrivalInformation_airport_code'
 - 'flightLegs_departureInformation_airport_code'
 - 'flightLegs_season'
 - 'flightLegs_arrivalInformation_times_scheduled_dayPeriod'
 - 'flightLegs_departureInformation_times_scheduled_dayPeriod'
 - Numériques
 - 'flightLegs_scheduledFlightDuration'

Hyperparameter tuning

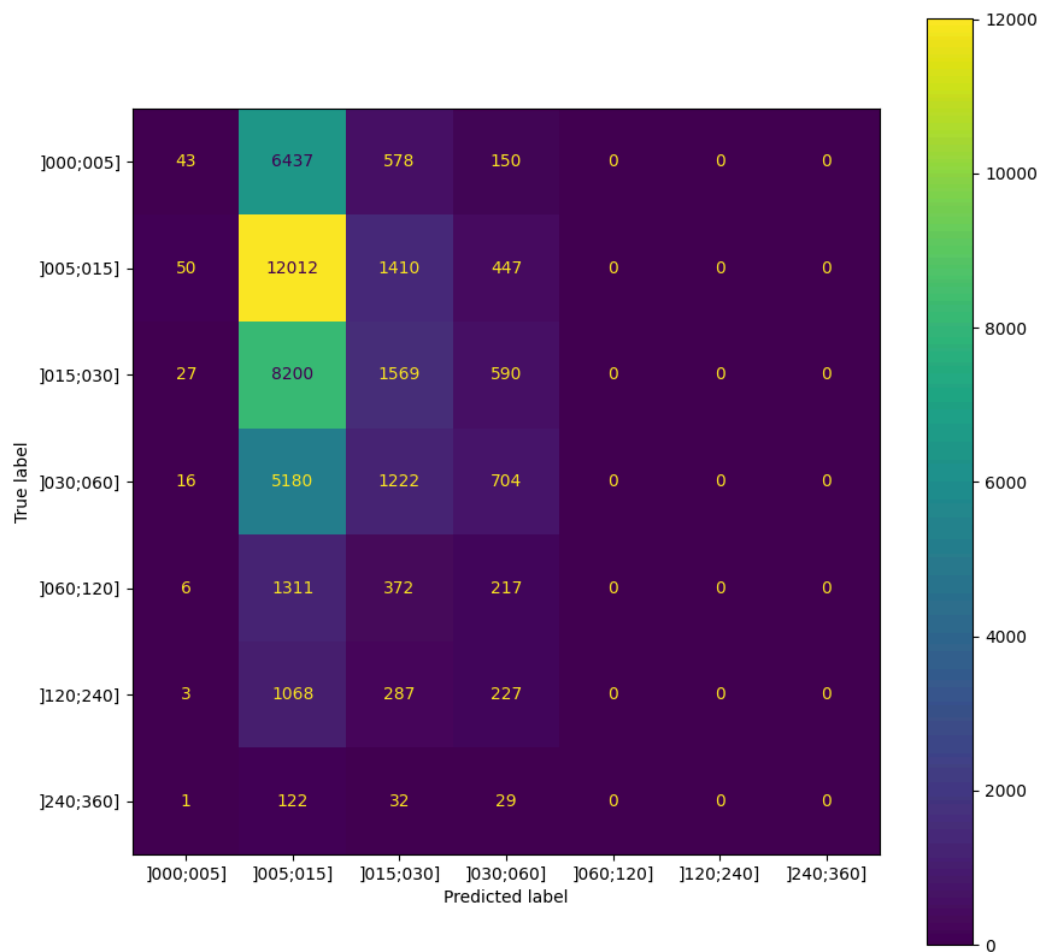
- GridSearchCV (petit nombre de paramètres pour tester le pipeline)
- cv = 5
- scoring = accuracy

Algorithmes

- Logistic_OVO (One vs One)
 - best score: 0.34
- Logistic_OVR (One vs Rest)
 - best score: 0.34
- DecisionTree
 - best score: 0.32
- RandomForest
 - best score: 0.32

Exemple de Matrice de confusion

Delay Duration - Logistic_OVR (One vs Rest)



Conclusion

Précision assez mauvaise (34%) même si un peu meilleure que pour le modèle de régression.

Durée du retard (régression)

Séparation jeux de données

- Vols avec flightLegs_irregularity_delayDuration_total > 0
- entraînement 80% > 169'000 vols
- test 20% > 42'000 vols

Type de données

- Valeur cible : flightLegs_irregularity_delayDuration_total
- Valeurs explicatives
 - Catégorielles
 - 'flightLegs_aircraft_typeCode'
 - 'flightLegs_arrivalInformation_airport_code'
 - 'flightLegs_departureInformation_airport_code'
 - 'flightLegs_season'
 - 'flightLegs_arrivalInformation_times_scheduled_dayPeriod'
 - 'flightLegs_departureInformation_times_scheduled_dayPeriod'
 - Numériques
 - 'flightLegs_scheduledFlightDuration'

Hyperparameter tuning

- GridSearchCV (petit nombre de paramètres pour tester la pipeline)
- cv = 5
- scoring = r²

Algorithmes

- LinearRegression
 - score: 0.05
- XGBRegressor
 - best score: 0.07
- DecisionTreeRegressor
 - best score: 0
 -
- RandomForestRegressor
 - best score: 0.02

Conclusion

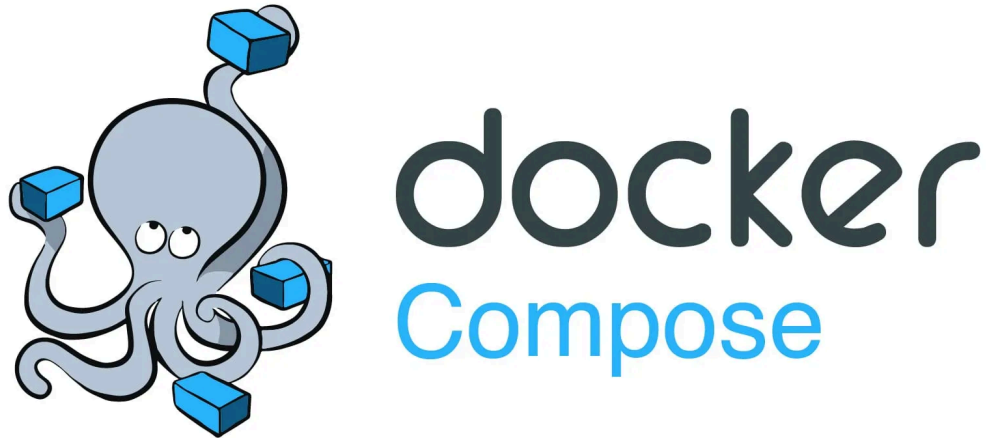
Algorithmes de prédiction de retard (classification et régression) mauvais sur ce jeu de donnée avec ces paramètres.

Pistes d'amélioration :

- Transformation de la variable cible si relations non linéaires avec les variables explicatives
- gestion des valeurs extrêmes
- autres algorithmes de machine learning
- Ajout/suppression de variables explicatives
- Hyperparameter tuning plus poussé

Étape 4 - Déploiement

Déploiement en local - Docker COMPOSE



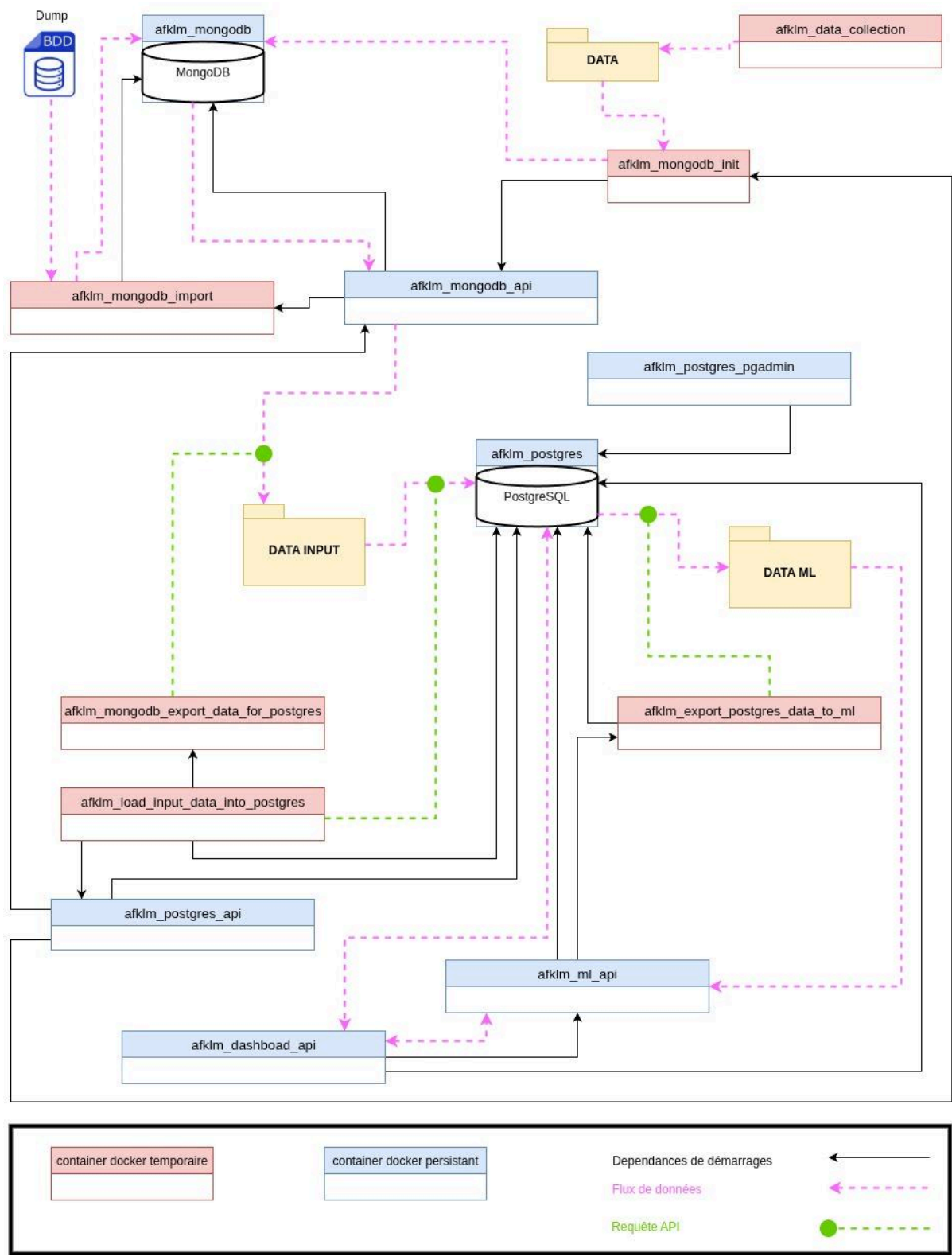
Docker nous permet le déploiement de nos services bdd, api, dashboards, etc. dans des conteneurs incluant tous les services nécessaires à leurs fonctionnements, évitant ainsi toute problématique de compatibilité du passage d'une machine de dev à une machine de déploiement (serveurs). L'ordonnancement des différents conteneurs Docker peut être organisé grâce à un fichier Docker Compose qui établit les dépendances entre services, les dossiers partagés/persistants et les variables d'environnement.

Pour notre projet, nous proposons un docker compose à la racine du projet donnant le choix à un utilisateur de lancer tout le projet en local, ou de le déployer facilement sur un serveur.

Deux types de conteneurs sont déployés par ce Docker Compose :

- Des conteneurs persistants hébergeant les différentes bases de données et APIs.
- Des conteneurs temporaires qui assurent les échanges de données entre les différents services (import de nouveaux vols dans MongoDB, export des csv depuis PostgreSQL, etc.) et qui sont arrêtés après accomplissement de leur tâche.

Structure des services et dépendances du Docker Compose



API - Apprentissage Machine

Air France KLM - ML API

0.1.0

OAS 3.1

[/openapi.json](#)

Flight delay prediction API

tests



GET /health Health Check



predictions



POST /predictions/get_delay_predictions Get Flight Delay Predictions



training



GET /retrain_models Retrain Models



GET /training_parameters_show Get Training Parameters



POST /training_parameters_upload Set Training Parameters



GET /training_parameters_defaults Reset Training Parameters



training_outputs



GET /display_training_run_list List Training Logs



GET /display_last_training_log Display Last Training Log



GET /download_best_models Download Best Models



metrics



GET /metrics/get_model_metrics Retrieve Metrics Of The Current Best Models



L'API associée au service d'apprentissage machine permet de gérer l'entraînement des modèles, consulter les paramètres/métriques et logs des meilleurs modèles entraînés et de d'obtenir une prévision de retard par le biais d'une requête POST fournissant les données de vol.

API - PostgreSQL

Air France KLM - PostgreSQL API

1.0.0 OAS 3.1

[/openapi.json](#)

REST API for updating the afklm PostgreSQL database with the latest data from MongoDB

tests

GET

/health

Check If The Api Is Running

⌵

training

GET

/download_historic_flights

Download Historic Flights

⌵

GET

/download_update_scheduled_d1_flights

Download Update D1 Flights

⌵

GET

/download_scheduled_flights

Download Scheduled Flights

⌵

GET

/load_mongodb_data_into_postgres

Loads The Contents Of The Files In The Data Input Folder Into The Postresql Database

⌵

GET

/export_all_collections_to_postgres_data_input

Export All Collections To Postgres Data Input

⌵

L'API associée au conteneur PostgreSQL permet télécharger/rapatrier les fichiers de données csv produits par MongoDB, de les importer dans PostgreSQL et de le traiter afin de peupler les différentes tables et vues de la base de donnée relationnelle.

API - mongoDB

REST API for querying and exporting Air France-KLM flight data from Europe's 30 largest airports



historic Query and export historic flight data with European airport filters		^
GET	/historic/with_id{id} Get Historic By Id	▼
GET	/historic/export Export Historic Flights	▼
scheduled Query and export scheduled flight data		^
GET	/scheduled/with_id{id} Get Scheduled By Id	▼
GET	/remove_scheduled/export Export Removed Scheduled	▼
GET	/scheduled/export Export Scheduled Flights	▼
scheduled d1 Query and export D1 scheduled flights (updated schedules)		^
GET	/update_scheduled_d1/with_id{id} Get Scheduled D1 By Id	▼
GET	/removed_scheduled_d1/export Export Removed D1 Flights	▼
GET	/update_scheduled_d1/export Export Scheduled D1 Flights	▼
database Database operations: statistics, exports, and dumps		^
GET	/collections/stats Get Collections Stats	▼
POST	/database/dumps Create Database Dump	▼



L'api MongoDB est l'interface permettant l'interaction du client avec la bdd mongoDB en assurant le respect des règles d'interaction et la sécurité entre ces deux parties. Faciliter l'interaction du client avec la BDD, protéger la BDD.

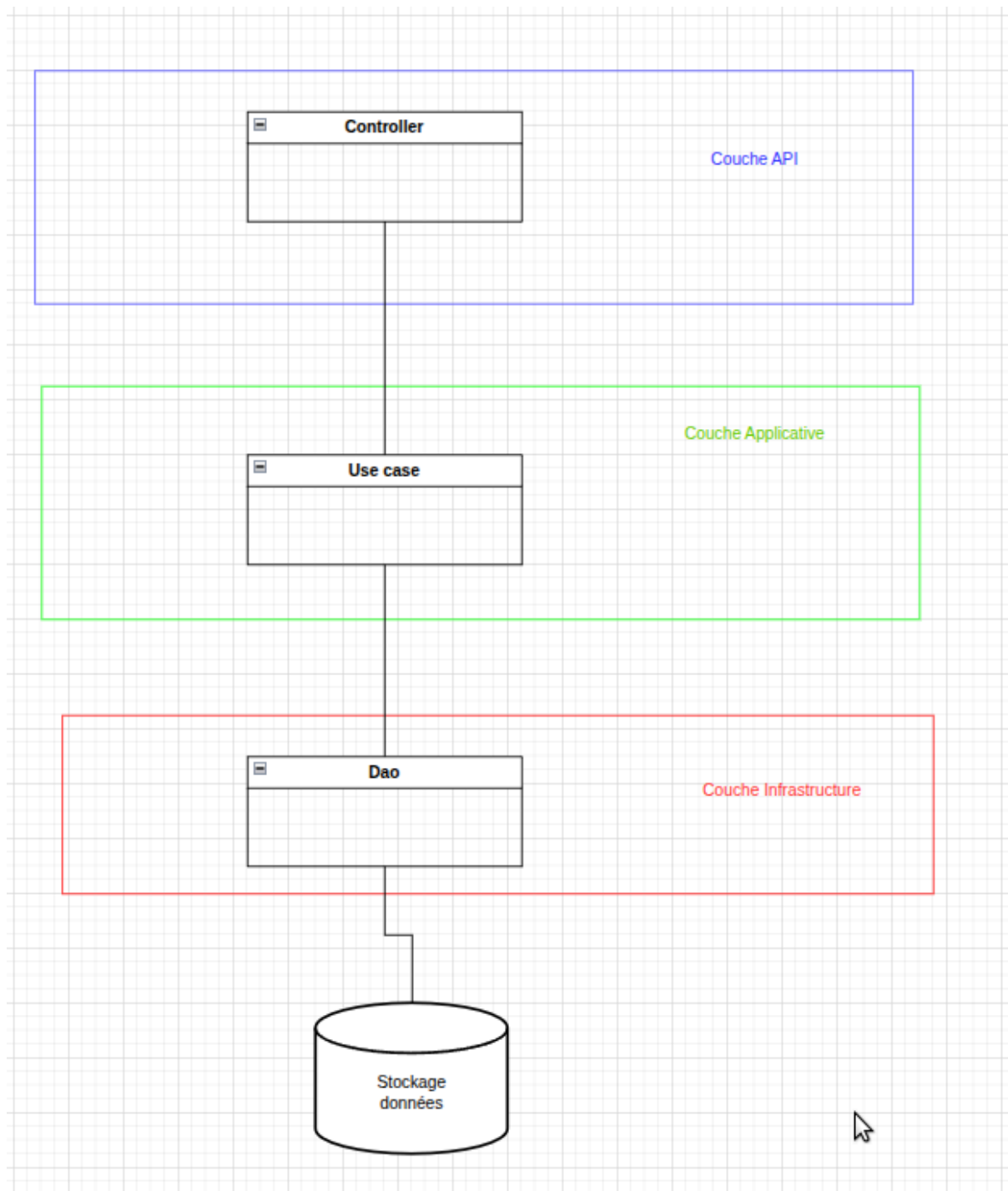
Elle permet de livrer aux clients des csv concernant les différentes collections que sont :

- scheduled
- scheduled d1
- historic

L'api propose aussi un dump et de savoir quel est les nombres de documents par collections dans la BDD.

Il manque d'ajouter OAuth2 pour protéger les routes car on a pas les droits pour redistribuer la donnée.

Architecture du code de l'API :



L'idée de cette architecture était de tenter de s'inspirer de la clean Architecture de Robert C.Martine. Bien que cela reste une tentative, l'idée était d'essayer au maximum de rendre indépendant la couche applicative (couche métier) de la BDD, ou du système de stockage de données, afin de pouvoir facilement le changer sans avoir à tout recoder. Le but était d'essayer au maximum de faire de l'inversion de dépendance. Cependant, pour faire quelque chose de plus dans les normes, il manque la couche domain qui contient les entités, il manque les DTO (Data transfert object), et surtout, la création d'interface pour chaque objet.

Couche API : Point d'entrée de l'application qui contient les controllers. Couche qui permet le routage vers la couche métier.

Couche Applicative : Il s'agit de la couche métiers, contenant les use case qui permettent de répondre à la demande client. (Client de l'API).

Couche Infrastructure : Couche qui contient les DAO (Data Access Object). Les Data Access Object. Dans le code de l'API du projet, c'est un abus de langage puisqu'on a pas de domain et donc pas d'objets à hydrater.

J'ai gardé le mot dans l'idée que les variables sont aussi des objets, et qu'ici les résultats de requêtes sont stockés dans des variables. Normalement, il aurait fallu créer une entité flight, la requête aurait dû hydrater cette entité.

Le DTO est ce qui devrait être retourné au client. Il manque donc la partie qui map les entités en DTO .

Le DTO attendu par le client dans notre cas est le fichier tabulaire. Le service qui transforme donc le flight Json en tabulaire, devrait être vue plus comme un mappeur qui map l'entité flight json, en tabulaire.

Interface utilisateur Dash

L'interface utilisateur du projet a été développée avec Plotly Dash, un framework Python spécialisé dans la création d'applications web interactives orientées data. L'objectif était de proposer une interface claire, ergonomique et entièrement dynamique pour l'exploration des données et la consultation des prédictions de retard de vols.

L'interface utilisateur inclut également un module FastAPI utilisant la méthode de requête POST pour envoyer une demande de prédiction sur le retard du vol sélectionné.

La connexion à la base de données Postgresql depuis Dash se fait avec la fonction `create_engine` de la librairie SQLAlchemy.

L'UI repose sur une architecture modulaire comprenant une sidebar interactive, un contenu principal adaptatif et plusieurs pages dédiées, grâce au système multipage natif de Dash (`use_pages=True`). La sidebar permet la navigation entre les différentes sections (tableau de bord, documentation, liens utiles...), tout en intégrant un système de retrait latéral (push) pour maximiser l'espace de travail.

Le cœur du dashboard intègre des composants Dash évolués tels que DataTable pour l'affichage filtrable et triable des données issues de PostgreSQL, ainsi que des cartes de visualisation pour les métriques modèles. Les styles visuels sont gérés via un fichier CSS personnalisé placé dans le dossier `assets/`, permettant l'ajout d'un thème sombre / clair, la gestion responsive des éléments, et la cohérence visuelle globale. L'ensemble forme une interface fluide, professionnelle et adaptée à une présentation de résultats de data science.



Prediction des retards

AF-KLM

API

MongoDB

Machine Learning

postgresql

pgAdmin

DOCUMENTATION

Récolte des données

Architecture des données

Consommation de la donnée

LIENS UTILES

GitHub du projet

Datascientiest

AUTEURS

Nicolas Calo

Johan Cloos

Rathana Lat

Younes Es-soualhi

Youssef Znati

THÈME

● Sombre

● Clair

Sélectionnez un vol dans le tableau ci-dessous pour obtenir une prédiction de retard.

Liste des vols à venir issus de la base PostgreSQL.

#	Flight id	Flight number	Airline name	Aircraft type code	Dep. continent name	Dep. subcontinent name	Dep. country name	Dep. airport name	Dep. code	Dep. data...
#	filter data...									
20251209AF+0001JFKDCG	1	AIR FRANCE	77W	Americas	North America	United States	John F. Kennedy International Airport	JFK	2025-	
20251209AF+0023LAXDCG	23	AIR FRANCE	77W	Americas	North America	United States	Los Angeles International Airport	LAX	2025-	
20251209AF+0031ATLDCG	31	AIR FRANCE	772	Americas	North America	United States	Hartsfield-Jackson Atlanta International Airport	ATL	2025-	
20251209AF+0137ORDCG	137	AIR FRANCE	359	Americas	North America	United States	Chicago O'Hare International Airport	ORD	2025-	
20251209AF+0140LOSDCG	149	AIR FRANCE	772	Africa	Western Africa	Nigeria	Murtala Muhammed International Airport	LOS	2025-	
20251209AF+0183BKKDCG	183	AIR FRANCE	772	Asia	Southeast Asia	Thailand	Suvarnabhumi Airport	BKK	2025-	
20251209AF+0191BLRDCG	191	AIR FRANCE	789	Asia	Southern Asia	India	Kempegowda International Airport	BLR	2025-	
20251209AF+041BCDGRHU	418	AIR FRANCE	77W	Europe	Western Europe	France	Paris Charles de Gaulle Airport	CDG	2025-	
20251209AF+071BCDGSSS	718	AIR FRANCE	77W	Europe	Western Europe	France	Paris Charles de Gaulle Airport	CDG	2025-	
20251209AF+0751PTTORY	751	AIR FRANCE	77W	Americas	Caribbean	Guadeloupe	Pointe-à-Pitre International Airport (Le Raizet Airport)	PTP	2025-	
20251209AF+0933LADCG	933	AIR FRANCE	332	Africa	Central Africa	Angola	Quatro de Fevereiro Airport	LAD	2025-	
20251209AF+0935TNMRCG	935	AIR FRANCE	772	Africa	Eastern Africa	Madagascar	Ivato International Airport	TNR	2025-	
20251209AF+1018CDGFR	1018	AIR FRANCE	E90	Europe	Western Europe	France	Paris Charles de Gaulle Airport	CDG	2025-	
20251209AF+1019FRACDG	1019	AIR FRANCE	E70	Europe	Central Europe	Germany	Frankfurt Airport	FRA	2025-	
20251209AF+1054CDGALG	1054	AIR FRANCE	320	Europe	Western Europe	France	Paris Charles de Gaulle Airport	CDG	2025-	
20251209AF+1059NCLCG	1059	AIR FRANCE	E90	Europe	Western Europe	Isle of Man	Newcastle Airport	NCL	2025-	
20251209AF+1060CDGDEV	1060	AIR FRANCE	320	Europe	Western Europe	France	Paris Charles de Gaulle Airport	CDG	2025-	

Sélectionnez un vol dans le tableau ci-dessous pour obtenir une prédiction de retard.

Liste des vols à venir issus de la base PostgreSQL

flight_id	flightnumber	airline_name	aircraft_typecode	dep_continent_name	dep_subcontinent_name	dep_country_name	dep_airport_name	dep_code	dep_date
20251210+AF+7509NTE									
20251210+AF+7509NTECG	7509	AIR FRANCE	223	Europe	Western Europe	France	Nantes Atlantique Airport	NTE	2025-12-10

Performances comparées des modèles de Machine Learning entraînés

pipeline	problem_type
Logistic_OVR	classification_delay
DecisionTree	classification_status
LinearRegression	regression

prediction_type	prediction_value
predicted_delay_min_classification	[005;015]
predicted_delay_min_regression	31.485714671206136
predicted_flight_status	LATE

Environnement GCP

Notre architecture s'appuie sur Google Cloud Platform (GCP) pour assurer l'extraction, le stockage et le traitement automatisé des données provenant de l'API Air France. Elle combine des services serverless, du stockage objet et des conteneurs afin de garantir une solution flexible, scalable et maintenable dans un contexte de production.

Stockage des données – Google Cloud Storage

Google Cloud Storage (GCS) est utilisé pour centraliser l'ensemble des données brutes collectées depuis l'API Air France. Les fichiers sont stockés dans des buckets GCS et deviennent la source de référence pour tous les traitements ultérieurs.

Rôle dans l'architecture :

- Point d'entrée du stockage des données
- Base de travail pour les étapes de transformation

Traitements automatisés – Cloud Run

Le cœur du traitement repose sur Cloud Run, qui exécute des services conteneurisés de façon entièrement serverless.

Fonctions Cloud Run

Ces fonctions orchestrent différentes étapes du pipeline :

- **Extraction_from_air_france_api :**
 - Extrait les données de vols depuis l'API Air France KLM
 - Stocke les données dans Google Cloud Storage
- **Extraction_gcs_to_mongodb :**
 - Charge les données depuis GCS vers MongoDB
 - Gère la déduplication et la validation

- **Populating_mongodb_future_postgre :**
 - Synchronise les vols futurs (collection `scheduled`) vers PostgreSQL
 - Table cible: `mongodb\_future`

- **populating_mongo_futured1_postgre :**
 - Synchronise les vols futurs D+1 (collection `update\_scheduled\_d1`) vers PostgreSQL
 - Table cible: `mongodb\_future\_d1`

- **Populating_mongodb_past :**
 - Synchronise les vols historiques (collection `historic`) vers PostgreSQL
 - Table cible: `mongodb\_past`

- **Clean_populate_fact_tables_in_postgre :**
 - Nettoie et agrège les données dans les tables de faits PostgreSQL
 - Applique les transformations business

Cloud Run

Les applications déployées sur Cloud Run sont:

- **mongodb_api :**
 - API FastAPI pour accéder aux données MongoDB
 - Endpoints: export CSV, requêtes personnalisées
 - Documentation: `/docs`
- **ml_api :**
 - API de Machine Learning pour prédictions
- **dash_app :**
 - Dashboard interactif pour la visualisation des données
 - Connexion PostgreSQL et MongoDB

Gestion des images – Cloud Build - Artifact Registry

Artifact Registry stocke les images Docker nécessaires aux différents services Cloud Run. Il fournit un espace de stockage sécurisé, prend en charge la gestion des versions et s'intègre directement avec les outils de déploiement GCP. Les images sont automatiquement construites par Cloud Build à partir d'un sous-répertoire du dépôt GitHub, puis publiées dans Artifact Registry pour être utilisées lors des déploiements Cloud Run.

Orchestration avancée – Google Kubernetes Engine (GKE)

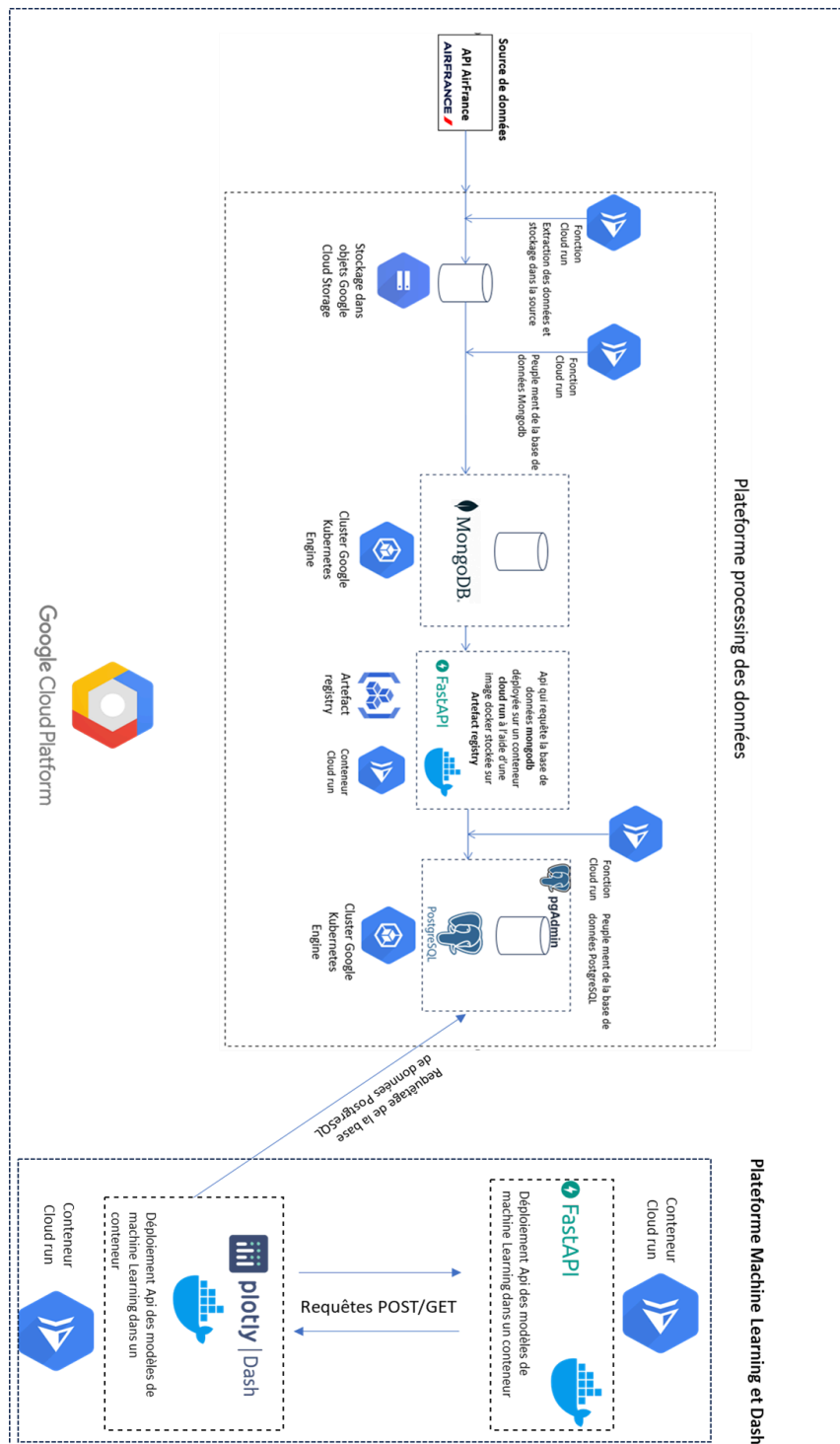
L'architecture repose sur deux clusters Google Kubernetes Engine distincts, dédiés respectivement aux bases de données MongoDB et PostgreSQL. Ce choix permet de séparer clairement les environnements afin de garantir une isolation optimale et d'éviter les risques de conflits ou de dégradation des performances. En dissociant les charges de travail, chaque cluster peut être configuré et dimensionné selon les besoins spécifiques de sa base de données, assurant ainsi une meilleure stabilité et une plus grande maîtrise des ressources.

Problèmes rencontrés lors du déploiement

Les principaux problèmes rencontrés lors du traitement des données provenaient d'une combinaison de facteurs techniques et de limites liées à l'environnement d'exécution. D'abord, des **erreurs de timeout** survenaient fréquemment : certains scripts nécessitent un temps d'exécution trop long en raison du volume de données à extraire de l'api et de la complexité des opérations (agrégations, jointures). À cela s'ajoutent des **restrictions de ressources** imposées par le budget d'essai, limitant la puissance de calcul disponible et accentuant les ralentissements. Par ailleurs, les **requêtes lourdes** effectuées pour transformer et migrer les données entre **MongoDB** et **PostgreSQL** sollicitent fortement les ressources, rendant l'ensemble du processus encore plus sensible aux dépassements de temps et aux blocages.

Pour résoudre ces difficultés, plusieurs optimisations ont été mises en place. Les scripts ont d'abord été **refactorisés**, avec une attention particulière portée à l'efficacité des fonctions, à la réduction du nombre d'itérations inutiles et à l'amélioration des boucles de traitement. Ensuite, un **traitement par batch** des documents issus de MongoDB a été introduit afin de diviser le flux de données en segments plus petits et plus faciles à gérer. Cette approche a permis d'éviter les pics de charge, de stabiliser les performances et d'assurer une exécution plus fluide, même dans un environnement à ressources limitées. Ensemble, ces optimisations ont contribué à rendre le processus de transformation et de migration plus fiable, plus rapide et mieux adapté aux contraintes du projet.

Architecture du projet sur GCP

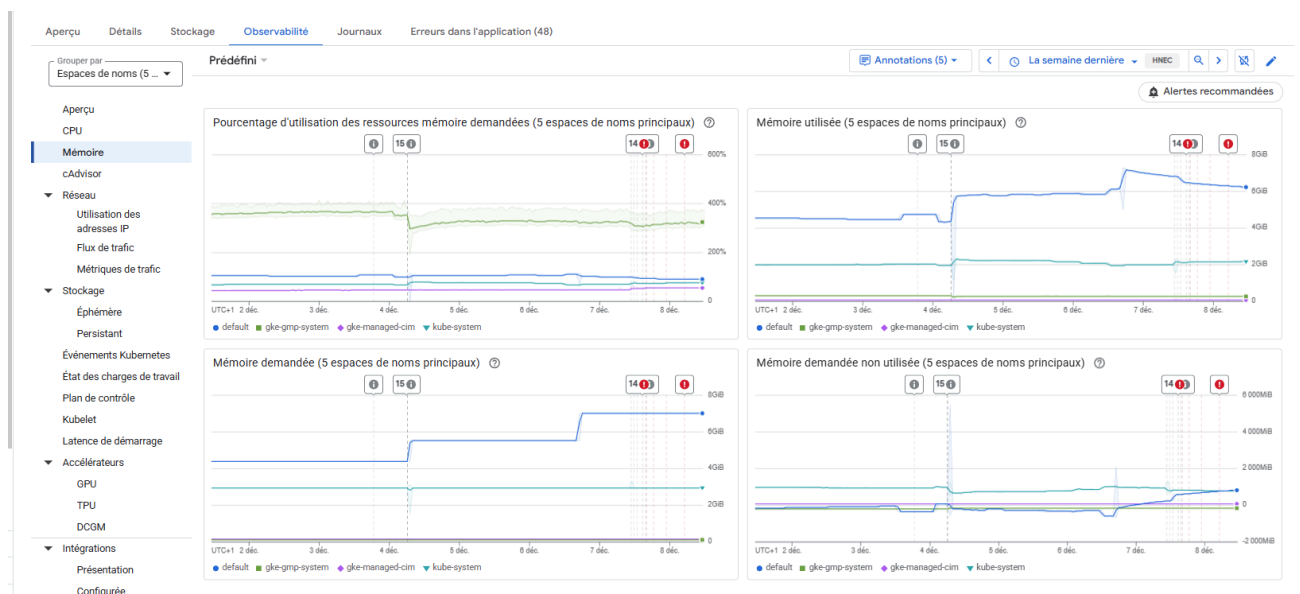
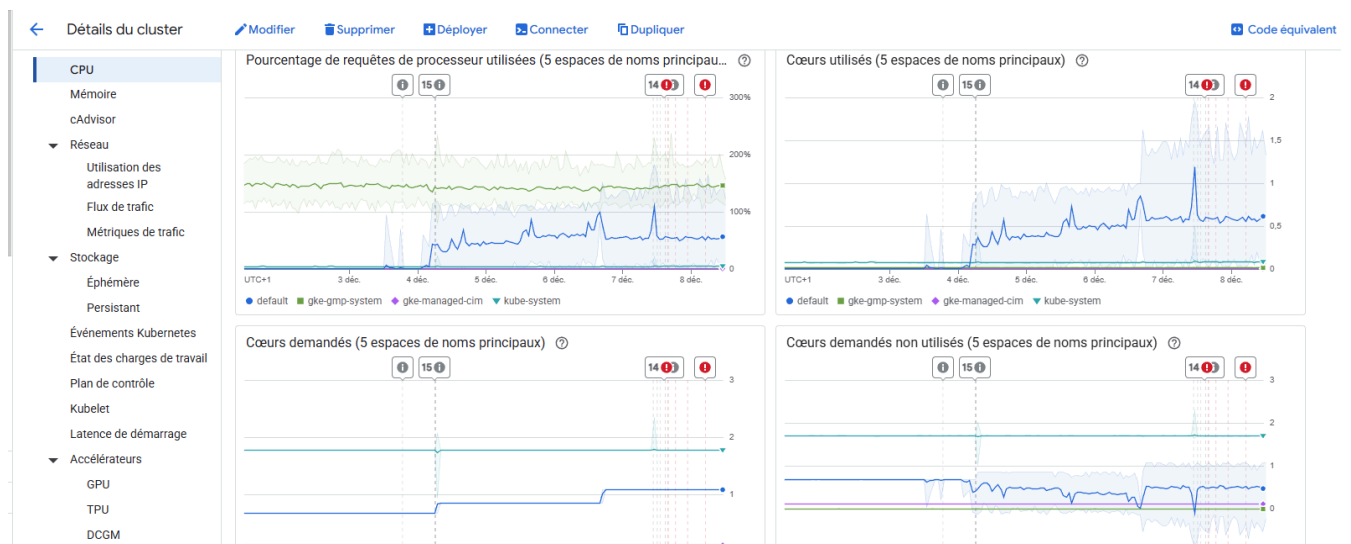


Étape 5 - Automatisation et Monitoring

Google Cloud Monitoring est l'outil centralisé de supervision proposé par GCP pour surveiller l'ensemble des ressources et services déployés dans le cloud. Il permet d'observer en temps réel l'état et les performances des applications Cloud Run, des Cloud Functions, ainsi que des charges de travail exécutées dans GKE. Grâce à la collecte automatique des métriques, des journaux et des traces, Cloud Monitoring offre une visibilité complète sur la santé des services, facilite l'identification des anomalies et accélère le diagnostic des incidents. Il permet également de configurer des tableaux de bord personnalisés, des alertes proactives et des intégrations avec d'autres outils d'observabilité, améliorant ainsi la fiabilité et la disponibilité des environnements cloud.

Dans notre projet, nous n'avons pas configuré pour le moment des mécanismes d'alerte sur les différentes applications déjà déployées, nous nous contentons d'analyser les logs et les métriques qu'on observe en sortie de Google Cloud Monitoring.

Tableau de bord pour le monitoring des composants GKE :



Logs fonction d'extractions :

Observabilité	Révisions	Source	Déclencheurs	Réseau	Sécurité	YAML
Métriques		Journaux	Niveau de gravité Par défaut	Filtrer Rechercher dans tous les champs et valeurs		
Journaux		Niveau de gravité	Horodatage	Résumé		
		> 2025-12-06 23:49:25.378 HNEC	call_parameter_lists/df_call_parameters_0a_euFromTo_single_airport_top30eu.csv			
SLO		> 2025-12-06 23:49:25.378 HNEC	#####			
		> 2025-12-06 23:49:54.264 HNEC	[INFO] 2025-12-06 22:49:54 - call_parameter_lists/df_call_parameters_0a_euFromTo_single_airport_top30eu.bak updated			
Erreurs		> 2025-12-06 23:49:54.264 HNEC	[INFO] 2025-12-06 22:49:54 - call_parameter_lists/df_call_parameters_0a_euFromTo_single_airport_top30eu.bak updated			
		> 2025-12-06 23:49:54.280 HNEC	[39mMax number of pages to retrieve: 1000000000			
		> 2025-12-06 23:49:54.280 HNEC	[INFO] 2025-12-06 22:49:54 - Max number of pages to retrieve: 1000000000			
		> 2025-12-06 23:49:54.280 HNEC	[INFO] 2025-12-06 22:49:54 - Max number of pages to retrieve: 1000000000			
		> 2025-12-06 23:49:54.280 HNEC	[39mNumber of API call parameters to process = 23200			
		> 2025-12-06 23:49:54.280 HNEC	[INFO] 2025-12-06 22:49:54 - Number of API call parameters to process = 23200			
		> 2025-12-06 23:49:54.280 HNEC	[INFO] 2025-12-06 22:49:54 - Number of API call parameters to process = 23200			
		> 2025-12-06 23:49:57.049 HNEC	[39morigin=LI&endRange=2025-11-29T23:59:59Z&startRange=2025-11-29T00:00:00Z			
		> 2025-12-06 23:49:57.049 HNEC	[INFO] 2025-12-06 22:49:57 - origin=LI&endRange=2025-11-29T23:59:59Z&startRange=2025-11-29T00:00:00Z			
		> 2025-12-06 23:49:57.050 HNEC	[INFO] 2025-12-06 22:49:57 - origin=LI&endRange=2025-11-29T23:59:59Z&startRange=2025-11-29T00:00:00Z			
		> 2025-12-06 23:49:57.051 HNEC	[35mskipped because previously failed due to server error			
		> 2025-12-06 23:49:57.051 HNEC	[WARNING] 2025-12-06 22:49:57 - skipped because previously failed due to server error			
		> 2025-12-06 23:49:57.052 HNEC	[WARNING] 2025-12-06 22:49:57 - skipped because previously failed due to server error			
		> 2025-12-06 23:50:08.101 HNEC	[39morigin=AMS&endRange=2026-03-21T23:59:59Z&startRange=2026-03-21T00:00:00Z			

Logs fonctions de peuplement de la base de données mongoDB

Observabilité	Révisions	Source	Déclencheurs	Réseau	Sécurité	YAML
Métriques		Journaux	Niveau de gravité Par défaut	⚙ Filtrer	Rechercher dans tous les champs et valeurs	
Journaux		Niveau de gravité	Horodatage	Résumé		
SLO		> 2025-12-08 11:06:29.163 HNEC	deleting duplicates			
		> 2025-12-08 11:06:29.219 HNEC	drop			
		> 2025-12-08 11:06:29.367 HNEC	move to collection scheduled_flights			
Erreurs		> 2025-12-08 11:06:29.463 HNEC	deleting duplicates			
		> 2025-12-08 11:06:30.958 HNEC	drop			
		> 2025-12-08 11:06:31.355 HNEC	nb duplicate scheduled_flights deleted : 0			
		> 2025-12-08 11:06:31.562 HNEC	nb d1 flights deleted : 0			
		> 2025-12-08 11:06:33.380 HNEC	nb scheduled flights deleted : 0			
		> 2025-12-08 11:06:33.583 HNEC	Nb docs on removed_update_scheduled_d1_flights updated to add date: 0			
		> 2025-12-08 11:06:34.619 HNEC	Nb docs on historic_flights updated to add date: 0			
		> 2025-12-08 11:06:34.721 HNEC	Nb docs on removed_scheduled_flights updated to add date: 0			
		> 2025-12-08 11:06:34.861 HNEC	Nb docs on update_scheduled_d1_flights updated to add date: 0			
		> 2025-12-08 11:06:35.171 HNEC	Nb docs on scheduled_flights updated to add date: 0			
		> 2025-12-08 11:06:35.371 HNEC	move to collection historic_flights			
		> 2025-12-08 11:06:35.433 HNEC	deleting duplicates			
		> 2025-12-08 11:06:44.662 HNEC	drop			
		> 2025-12-08 11:06:44.772 HNEC	move to collection update_scheduled_d1_flights			
		> 2025-12-08 11:06:44.838 HNEC	deleting duplicates			
		> 2025-12-08 11:06:44.857 HNEC	drop			
		> 2025-12-08 11:06:44.964 HNEC	move to collection scheduled_flights			
		> 2025-12-08 11:06:45.048 HNEC	deleting duplicates			
		> 2025-12-08 11:06:46.288 HNEC	drop			
		> 2025-12-08 11:06:46.607 HNEC	nb duplicate scheduled_flights deleted : 0			
		> 2025-12-08 11:06:46.745 HNEC	nb d1 flights deleted : 0			
		> 2025-12-08 11:06:48.459 HNEC	nb scheduled flights deleted : 0			
		> 2025-12-08 11:06:48.580 HNEC	Nb docs on removed_update_scheduled_d1_flights updated to add date: 0			
		> 2025-12-08 11:06:49.577 HNEC	Nb docs on historic_flights updated to add date: 0			

logs transformation des données dans postgresQL

Observabilité	Révisions	Source	Déclencheurs	Réseau	Sécurité	YAML
Métriques		Journaux	Niveau de gravité Par défaut	Filtrer Rechercher dans tous les champs et valeurs		
Journaux		Niveau de gravité	Hierarchage	Résumé		
SLO		2025-12-06 16:00:04.531 HNEC	[INFO]	2025-12-06 15:00:04	Executing: INSERT INTO departure_future (flight_id, flightlegs_depInfo_airport_code, flight...	
		2025-12-06 16:00:04.545 HNEC	[INFO]	2025-12-06 15:00:04	SQL execution successful.	
Erreurs		2025-12-06 16:00:04.545 HNEC	[INFO]	2025-12-06 15:00:04	Executing: INSERT INTO arrival_future (flight_id, flightlegs_arrInfo_airport_code, flightle...	
		2025-12-06 16:00:04.551 HNEC	[INFO]	2025-12-06 15:00:04	SQL execution successful.	
		2025-12-06 16:00:04.555 HNEC	[INFO]	2025-12-06 15:00:04	SQL execution finished.	
		2025-12-06 16:00:04.555 HNEC	[INFO]	2025-12-06 15:00:04	insert select flight future done successfully	
		2025-12-06 16:00:04.572 HNEC	[INFO]	2025-12-06 15:00:04	Running SQL string...	
		2025-12-06 16:00:04.572 HNEC	[INFO]	2025-12-06 15:00:04	Executing: DELETE FROM flight_future USING flight_past	
		2025-12-06 16:00:04.572 HNEC			WHERE flight_future.flight_id = f1...	
		2025-12-06 16:00:04.583 HNEC	[INFO]	2025-12-06 15:00:04	SQL execution successful.	
		2025-12-06 16:00:04.583 HNEC	[INFO]	2025-12-06 15:00:04	Executing: DELETE FROM arrival_future USING arrival_past	
		2025-12-06 16:00:04.583 HNEC			WHERE arrival_future.flight_id = *...	
		2025-12-06 16:00:04.589 HNEC	[INFO]	2025-12-06 15:00:04	SQL execution successful.	
		2025-12-06 16:00:04.589 HNEC	[INFO]	2025-12-06 15:00:04	Executing: DELETE FROM departure_future USING departure_past	
		2025-12-06 16:00:04.589 HNEC			WHERE departure_future.flight...	
		2025-12-06 16:00:04.594 HNEC	[INFO]	2025-12-06 15:00:04	SQL execution successful.	
		2025-12-06 16:00:04.596 HNEC	[INFO]	2025-12-06 15:00:04	SQL execution finished.	
		2025-12-06 16:00:04.596 HNEC	[INFO]	2025-12-06 15:00:04	removing from future based on past done successfully	
		2025-12-06 16:00:04.611 HNEC	[INFO]	2025-12-06 15:00:04	Running SQL string...	
		2025-12-06 16:00:04.611 HNEC	[INFO]	2025-12-06 15:00:04	Executing: DROP TABLE mongodb_past...	
		2025-12-06 16:00:04.620 HNEC	[INFO]	2025-12-06 15:00:04	SQL execution successful.	
		2025-12-06 16:00:04.620 HNEC	[INFO]	2025-12-06 15:00:04	Executing: DROP TABLE mongodb_future...	
		2025-12-06 16:00:04.625 HNEC	[INFO]	2025-12-06 15:00:04	SQL execution successful.	
		2025-12-06 16:00:04.629 HNEC	[INFO]	2025-12-06 15:00:04	SQL execution finished.	
		2025-12-06 16:00:04.630 HNEC	[INFO]	2025-12-06 15:00:04	final cleaning done successfully	

Logs API Mongodb

Observabilité	Révisions	Source	Déclencheurs	Réseau	Sécurité	YAML
Métriques	Niveau de gravité Par défaut Filter Rechercher dans tous les champs et valeurs					
Journaux	Niveau de gravité	Horodatage	Résumé			
SLO						
Erreurs						

Logs API machine learning

Journaux

Niveau de gravité

Par défaut

Filtrer

Rechercher dans tous les champs et valeurs

Niveau de gravité	Horodatage	Résumé
>	2025-12-08 01:14:57.011 HNEC	INFO: 169.254.169.126:40166 - "POST /get_delay_predictions HTTP/1.1" 200 OK
>	2025-12-08 01:14:57.848 HNEC	POST 200 220 o 21 ms python-requests/2.32.5 https://mongo-db-901450845940.europe-west1.run.app/get_delay_predictions
>	2025-12-08 01:14:57.867 HNEC	/usr/local/lib/python3.12/site-packages/sklearn/preprocessing/_encoders.py:246: UserWarning: Found unknown categories in columns [0, 2, 3, 4, 6] during transform. These unknown categories will be encoded as all_
>	2025-12-08 01:14:57.867 HNEC	warnings.warn()
>	2025-12-08 01:14:57.869 HNEC	INFO: 169.254.169.126:40182 - "POST /get_delay_predictions HTTP/1.1" 200 OK
>	2025-12-08 01:14:58.903 HNEC	POST 200 220 o 20 ms python-requests/2.32.5 https://mongo-db-901450845940.europe-west1.run.app/get_delay_predictions
>	2025-12-08 01:14:58.921 HNEC	/usr/local/lib/python3.12/site-packages/sklearn/preprocessing/_encoders.py:246: UserWarning: Found unknown categories in columns [2, 3, 4, 6] during transform. These unknown categories will be encoded as all_
>	2025-12-08 01:14:58.922 HNEC	warnings.warn()
>	2025-12-08 01:14:58.924 HNEC	INFO: 169.254.169.126:40198 - "POST /get_delay_predictions HTTP/1.1" 200 OK
>	2025-12-08 01:17:12.134 HNEC	GET 200 4,74 ko 4 ms Chrome 143 https://mongo-db-901450845940.europe-west1.run.app/model_parameters_and_metrics
>	2025-12-08 01:17:12.139 HNEC	INFO: 169.254.169.126:45264 - "GET /model_parameters_and_metrics HTTP/1.1" 200 OK
>	2025-12-08 01:17:13.891 HNEC	GET 200 4,74 ko 3 ms Chrome 143 https://mongo-db-901450845940.europe-west1.run.app/model_parameters_and_metrics
>	2025-12-08 01:17:13.895 HNEC	INFO: 169.254.169.126:45270 - "GET /model_parameters_and_metrics HTTP/1.1" 200 OK
>	2025-12-08 01:17:25.435 HNEC	POST 200 220 o 22 ms python-requests/2.32.5 https://mongo-db-901450845940.europe-west1.run.app/get_delay_predictions
>	2025-12-08 01:17:25.457 HNEC	/usr/local/lib/python3.12/site-packages/sklearn/preprocessing/_encoders.py:246: UserWarning: Found unknown categories in columns [2, 3, 4, 6] during transform. These unknown categories will be encoded as all_
>	2025-12-08 01:17:25.457 HNEC	warnings.warn()
>	2025-12-08 01:17:25.459 HNEC	INFO: 169.254.169.126:37688 - "POST /get_delay_predictions HTTP/1.1" 200 OK
>	2025-12-08 01:17:29.581 HNEC	POST 200 220 o 21 ms python-requests/2.32.5 https://mongo-db-901450845940.europe-west1.run.app/get_delay_predictions
>	2025-12-08 01:17:29.600 HNEC	/usr/local/lib/python3.12/site-packages/sklearn/preprocessing/_encoders.py:246: UserWarning: Found unknown categories in columns [0, 2, 3, 4, 6] during transform. These unknown categories will be encoded as all_
>	2025-12-08 01:17:29.600 HNEC	warnings.warn()
>	2025-12-08 01:17:29.603 HNEC	INFO: 169.254.169.126:14018 - "POST /get_delay_predictions HTTP/1.1" 200 OK
>	2025-12-08 01:18:12.821 HNEC	POST 200 220 o 22 ms python-requests/2.32.5 https://mongo-db-901450845940.europe-west1.run.app/get_delay_predictions
>	2025-12-08 01:18:12.842 HNEC	/usr/local/lib/python3.12/site-packages/sklearn/preprocessing/_encoders.py:246: UserWarning: Found unknown categories in columns [0, 2, 3, 4, 6] during transform. These unknown categories will be encoded as all_
>	2025-12-08 01:18:12.842 HNEC	warnings.warn()
>	2025-12-08 01:18:12.845 HNEC	INFO: 169.254.169.126:3176 - "POST /get_delay_predictions HTTP/1.1" 200 OK

Annexe

Diagramme UML complet généré avec Dbeaver

